Programación de bases de datos.

Caso práctico

Juan recuerda, de cuando estudió el Ciclo de Desarrollo de Aplicaciones Informáticos, que había muchas tareas que se podían automatizar dentro de la base de datos mediante el uso de un lenguaje de programación, e incluso que se podían programar algunas restricciones a la hora de manipular los datos. **Juan** se lo comenta a **María** y ésta se muestra ilusionada con dicha idea ya que muchas veces repiten el trabajo con la base de datos de juegos on-line que tienen entre manos (consultas, inserciones, etc. que son muy parecidas y que se podrían automatizar).

Para ello, hablan con **Ada** y ésta les comenta que claro que se puede hacer y que precisamente eso es lo que les toca hacer ahora. **Ada** les dice que para ese propósito existe un lenguaje de programación llamado PL/SQL que permite hacer lo que ellos quieren, así que les pasa un manual para que se lo vayan leyendo y se vayan poniendo manos a la obra con la base de datos de juegos on-line.

Ahora que ya dominas el uso de SQL para la manipulación y consulta de datos, es el momento de dar una vuelta de tuerca adicional para mejorar las aplicaciones que utilicen nuestra base de datos. Para ello nos vamos a centrar en la programación de bases de datos, utilizando el lenguaje PL/SQL. En esta unidad conoceremos qué es PL/SQL, cuál es su sintaxis y veremos cómo podemos sacarle el máximo partido a nuestra base de datos mediante su uso.

Debes conocer

La mayor parte de los ejemplos de esta unidad están basados en el modelo de datos extraído del siguiente caso de estudio:

Caso de estudio.

1.- Introducción.

Caso práctico

Juan y **María** se han puesto a repasar el manual de PL/SQL que les ha pasado Ada. Aunque no han avanzado mucho con el mismo, ya saben a qué se van a enfrentar y los beneficios que pueden obtener del uso del mismo para su aplicación de juegos on-line. Cuando hacen la primera parada de la mañana para tomarse un café, ambos se ponen a comentar las primeras conclusiones que han sacado después de su primer acercamiento a este lenguaje. Ambos están deseosos de seguir avanzando en su aprendizaje y saben que para ello cuentan con la inestimable ayuda de **Ada**.

Estarás pensado que si no tenemos bastante con aprender SQL, sino que ahora tenemos que aprender otro lenguaje más que lo único que va a hacer es complicarnos la vida. Verás que eso no es cierto ya que lo más importante, que es el conocimiento de SQL, ya lo tienes. PL/SQL tiene una sintaxis muy sencilla y verás cómo pronto te acostumbras y luego no podrás vivir sin él.

Pero, **¿qué es realmente PL/SQL?**

PL/SQL es un lenguaje procedimental estructurado en bloques que amplía la funcionalidad de SQL. Con PL/SQL podemos usar sentencias SQL para manipular datos y sentencias de control de flujo para procesar los datos. Por tanto, PL/SQL combina la potencia de SQL para la manipulación de datos, con la potencia de los lenguajes procedimentales para procesar los datos.

Aunque PL/SQL fue creado por Oracle, hoy día todos los gestores de bases de datos utilizan un lenguaje procedimental muy parecido al ideado por Oracle para poder programar las bases de datos.

Como veremos, en PL/SQL podemos definir variables, constantes, funciones, procedimientos, capturar errores en tiempo de ejecución, anidar cualquier número de bloques, etc. como solemos hacer en cualquier otro lenguaje de programación. Además, por medio de PL/SQL programaremos los disparadores de nuestra base de datos, tarea que no podríamos hacer sólo con SQL.

El motor de PL/SQL acepta como entrada bloques PL/SQL o subprogramas, ejecuta sentencias procedimentales y envía sentencias SQL al servidor de bases de datos. En el esquema adjunto puedes ver su funcionamiento.

Una de las grandes ventajas que nos ofrece PL/SQL es un mejor rendimiento en entornos de red cliente-servidor, ya que permite mandar bloques PL/SQL desde el cliente al servidor a través de la red, reduciendo de esta forma el tráfico y así no tener que mandar una a una las sentencias SQL correspondientes.

EJECUCIÓN DE PL/SQL
Bloque PL/SQL
Motor PL/SQL
Sentencias SQL
Motor SQL
Base de Datos

Para saber más

En el siguiente enlace podrás encontrar una breve historia de PL/SQL.

Historia de PL/SQL.

En estos enlaces podrás comprobar como los gestores de bases de datos incluyen hoy día un lenguaje procedimental para programar la base de datos muy parecido a PL/SQL.

Procedimientos almacenados en MySQL.

Lenguaje procedimental en PostgreSQL.

2.- Conceptos básicos.

Caso práctico

Juan ha avanzado muy rápido en la lectura del manual que le pasó **Ada**. **Juan** ya sabe programar en otros lenguajes de programación y por tanto la lectura de los primeros capítulos que se centran en cómo se estructura el lenguaje, los tipos de datos, las estructuras de control, etc. le han resultado muy fáciles de comprender. Sabe que lo único que tendrá que tener en cuenta son algunos aspectos en cuanto a las reglas de escritura y demás, pero la lógica es la de cualquier otro lenguaje de programación.

Como **María** está un poco más verde en el tema de la programación, **Juan** se ha ofrecido a darle un repaso rápido a todo lo que él ha visto y a explicarle todo aquello en lo que tenga dudas y a ver si pronto se pueden poner manos a la obra con la base de datos de juegos on-line.

En este apartado nos vamos a ir introduciendo poco a poco en los diferentes conceptos que debemos tener claros para programar en PL/SQL. Como para cualquier otro lenguaje de programación, debemos conocer las reglas de sintaxis que podemos utilizar, los diferentes elementos de que consta, los tipos de datos de los que disponemos, las estructuras de control que nos ofrece (tanto iterativas como condicionales) y cómo se realiza el manejo de los errores.

Como podrás comprobar, es todo muy sencillo y pronto estaremos escribiendo fragmentos de código que realizan alguna tarea particular. ¡Vamos a ello!

Autoevaluación

Indica cuáles de las siguientes características que nos proporciona PL/SQL son ciertas.

☐ Permite reducir el tráfico en la red en entornos cliente-servidor.
☐ No podemos utilizar sentencias SQL dentro de un bloque PL/SQL.
☐ Nos ofrece las ventajas de SQL y la potencia de un lenguaje procedimental.
☐ Para utilizar PL/SQL debemos instalar diferentes drivers en nuestra base de datos Oracle.

Mostrar Información

2.1.- Unidades léxicas (I).

En este apartado nos vamos a centrar en conocer cuáles son las unidades léxicas que podemos utilizar para escribir código en PL/SQL. Al igual que en nuestra lengua podemos distinguir diferentes unidades léxicas como palabras, signos de puntuación, etc. En los lenguajes de programación también existen diferentes unidades léxicas que definen los elementos más pequeños que tienen sentido propio y que al combinarlos de manera adecuada, siguiendo las reglas de sintaxis, dan lugar a sentencias válidas sintácticamente.

PL/SQL es un lenguaje no sensible a las mayúsculas, por lo que será equivalente escribir en mayúsculas o minúsculas, excepto cuando hablemos de literales de tipo cadena o de tipo carácter.

Cada unidad léxica puede estar separada por espacios (debe estar separada por espacios si se trata de 2 identificadores), por saltos de línea o por tabuladores para aumentar la legibilidad del código escrito.

```
IF A=CLAVE THEN ENCONTRADO:=TRUE;ELSE ENCONTRADO:=FALSE;END IF;
```

Sería equivalente a escribir la siguiente línea:

```
if a=clave then encontrado:=true;else encontrado:=false;end if;
```

Y también sería equivalente a este otro fragmento:

```
IF a = clave THEN
    encontrado := TRUE;
ELSE
    encontrado := FALSE;
END IF;
```

Las unidades léxicas se pueden clasificar en:

- ✓ Delimitadores.
- ✓ Identificadores.
- ✓ Literales.
- ✓ Comentarios.

Vamos a verlas más detenidamente.

Delimitadores.

PL/SQL tiene un conjunto de símbolos denominados delimitadores utilizados para representar operaciones entre tipos de datos, delimitar comentarios, etc. En la siguiente tabla puedes ver un resumen de los mismos.

Delimitadores en PL/SQL.

Delimitadores Simples.		Delimitadores Compuestos.	
Símbolo.	**Significado.**	**Símbolo.**	**Significado.**
+	Suma.	**	Exponenciación.
%	Indicador de atributo.	<>	Distinto.
.	Selector.	!=	Distinto.
/	División.	<=	Menor o igual.
(	Delimitador de lista.	>=	Mayor o igual.
)	Delimitador de lista.	..	Rango.
:	Variable host.	||	Concatenación.
,	Separador de elementos.	<<	Delimitador de etiquetas.
*	Producto.	>>	Delimitador de etiquetas.
"	Delimitador de identificador acotado.	--	Comentario de una línea.
=	Igual relacional.	/*	Comentario de varias líneas.
<	Menor.	*/	Comentario de varias líneas.
>	Mayor.	:=	Asignación.
@	Indicador de acceso remoto.	=>	Selector de nombre de parámetro.
;	Terminador de sentencias.		
-	Resta/negación.		

2.1.1.- Unidades léxicas (II).

Ya hemos visto qué son los delimitadores. Ahora vamos a continuar viendo el resto de unidades léxicas que nos podemos encontrar en PL/SQL.

Identificadores.

Los identificadores en PL/SQL, como en cualquier otro lenguaje de programación, son utilizados para nombrar elementos de nuestros programas. A la hora de utilizar los identificadores debemos tener en cuenta los siguientes aspectos:

- Un identificador es una letra seguida opcionalmente de letras, números, $, _, #.
- No podemos utilizar como identificador una palabra reservada.
  - Ejemplos válidos: **X, A1, codigo_postal**.
  - Ejemplos no válidos: **rock&roll, on/off**.
- PL/SQL nos permite además definir los identificadores acotados, en los que podemos usar cualquier carácter con una longitud máxima de 30 y deben estar delimitados por ". Ejemplo: **"X*Y"**.
- En PL/SQL existen algunos identificadores predefinidos y que tienen un significado especial ya que nos permitirán darle sentido sintáctico a nuestros programas. Estos identificadores son las palabras reservadas y no las podemos utilizar como identificadores en nuestros programas. Ejemplo: **IF, THEN, ELSE** ...
- Algunas palabras reservadas para PL/SQL no lo son para SQL, por lo que podríamos tener una tabla con una columna llamada '**type**' por ejemplo, que nos daría un error de compilación al referirnos a ella en PL/SQL. La solución sería acotarlos. **SELECT "TYPE"** ...

Literales.

Los literales se utilizan en las comparaciones de valores o para asignar valores concretos a los identificadores que actúan como variables o constantes. Para expresar estos literales tendremos en cuenta que:

- Los literales numéricos se expresarán por medio de notación decimal o de notación exponencial. Ejemplos: 234, +341, 2e3, -2E-3, 7.45, 8.1e3.
- Los literales tipo carácter y tipo cadena se deben delimitar con unas comillas simples.
- Los literales lógicos son **TRUE** y **FALSE**.
- El literal **NULL** que expresa que una variable no tiene ningún valor asignado.

Comentarios.

En los lenguajes de programación es muy conveniente utilizar comentarios en mitad del código. Los comentarios no tienen ningún efecto sobre el código pero sí ayudan mucho al programador o la programadora a recordar qué se está intentando hacer en cada caso (más aún cuando el código es compartido entre varias personas que se dedican a mejorarlo o corregirlo).

En PL/SQL podemos utilizar dos tipos de comentarios:

- Los comentarios de una línea se expresaran por medio del delimitador --. Ejemplo:

```
a:=b; --asignación
```

- Los comentarios de varias líneas se acotarán por medio de los delimitadores /* y */. Ejemplo:

```
/* Primera línea de comentarios.
   Segunda línea de comentarios. */
```

Ejercicio resuelto

Dada la siguiente línea de código, haz su descomposición en las diferentes unidades léxicas que contenga.

```
IF A <> B THEN iguales := FALSE; --No son iguales
```

2.2.- Tipos de datos simples, variables y constantes.

En cualquier lenguaje de programación, las variables y las constantes tienen un tipo de dato asignado (bien sea explícitamente o implícitamente). Dependiendo del tipo de dato el lenguaje de programación sabrá la estructura que utilizará para su almacenamiento, las restricciones en los valores que puede aceptar, la precisión del mismo, etc.

En PL/SQL contamos con todos los **tipos de datos simples** utilizados en SQL y algunos más. En este apartado vamos a enumerar los más utilizados.

Numéricos.

- ✓ **BINARY_INTEGER**: Tipo de dato numérico cuyo rango es de -2147483647 .. 2147483647. PL/SQL además define algunos subtipos de éste: **NATURAL**, **NATURALN**, **POSITIVE**, **POSITIVEN**, **SIGNTYPE**.
- ✓ **NUMBER**: Tipo de dato numérico para almacenar números racionales. Podemos especificar su escala (-84 .. 127) y su precisión (1 .. 38). La escala indica cuándo se redondea y hacia dónde. Ejemplos. escala=2: 8.234 -> 8.23, escala=-3: 7689 -> 8000. PL/SQL también define algunos subtipos como: **DEC**, **DECIMAL**, **DOUBLE PRECISION**, **FLOAT**, **INTEGER**, **INT**, **NUMERIC**, **REAL**, **SMALLINT**.
- ✓ **PLS_INTEGER**: Tipo de datos numérico cuyo rango es el mismo que el del tipo de dato **BINARY_INTEGER**, pero que su representación es distinta por lo que las operaciones aritméticas llevadas a cabo con los mismos serán más eficientes que en los dos casos anteriores.

Alfanuméricos.

- ✓ **CHAR(n)**: Array de n caracteres, máximo 2000 bytes. Si no especificamos longitud sería 1.
- ✓ **LONG**: Array de caracteres con un máximo de 32760 bytes.
- ✓ **RAW**: Array de bytes con un número máximo de 2000.
- ✓ **LONG RAW**: Array de bytes con un máximo de 32760.
- ✓ **VARCHAR2**: Tipo de dato para almacenar cadenas de longitud variable con un máximo de 32760.

Grandes objetos.

- ✓ **BFILE**: Puntero a un fichero del Sistema Operativo.
- ✓ **BLOB**: Objeto binario con una capacidad de 4 GB.
- ✓ **CLOB**: Objeto carácter con una capacidad de 2 GB.

Otros.

- ✓ **BOOLEAN**: **TRUE/FALSE**.
- ✓ **DATE**: Tipo de dato para almacenar valores día/hora desde el 1 enero de 4712 a.C. hasta el 31 diciembre de 4712 d.C.

Hemos visto los tipos de datos simples más usuales. Los tipos de datos compuestos los dejaremos para posteriores apartados.

Para saber más

En el siguiente enlace podrás ampliar información sobre los tipos de datos de los que disponemos en PL/SQL.

Tipos de datos en PL/SQL.

Autoevaluación

En PL/SQL cuando vamos a trabajar con enteros es preferible utilizar el tipo de dato BINARY_INTEGER, en vez de PLS_INTEGER.

○ Verdadero.
○ Falso.

2.2.1.- Subtipos.

Cuántas veces no has deseado cambiarle el nombre a las cosas por alguno más común para ti. Precisamente, esa es la posibilidad que nos ofrece PL/SQL con la utilización de los subtipos.

PL/SQL nos permite definir subtipos de tipos de datos para darles un nombre diferente y así aumentar la legibilidad de nuestros programas. Los tipos de operaciones aplicables a estos subtipos serán las mismas que los tipos de datos de los que proceden. La sintaxis será:

```
SUBTYPE subtipo IS tipo_base;
```

Donde subtipo será el nombre que le demos a nuestro subtipo y tipo_base será cualquier tipo de dato en PL/SQL.

A la hora de especificar el tipo base, podemos utilizar el modificador %TYPE para indicar el tipo de dato de una variable o de una columna de la base de datos y %ROWTYPE para especificar el tipo de un cursor o tabla de una base de datos.

```
SUBTYPE id_familia IS familias.identificador%TYPE;
SUBTYPE agente IS agentes%ROWTYPE;
```

Los subtipos no podemos restringirlos, pero podemos usar un truco para conseguir el mismo efecto y es por medio de una variable auxiliar:

```
SUBTYPE apodo IS varchar2(20);          --ilegal
aux varchar2(20);
SUBTYPE apodo IS aux%TYPE;               --legal
```

Los subtipos son intercambiables con su tipo base. También son intercambiables si tienen el mismo tipo base o si su tipo base pertenece a la misma familia:

```
DECLARE
    SUBTYPE numero IS NUMBER;
    numero_tres_digitos NUMBER(3);
    mi_numero_de_la_suerte numero;
    SUBTYPE encontrado IS BOOLEAN;
    SUBTYPE resultado IS BOOLEAN;
    lo_he_encontrado encontrado;
    resultado_busqueda resultado;
    SUBTYPE literal IS CHAR;
    SUBTYPE sentencia IS VARCHAR2;
    literal_nulo literal;
    sentencia_vacia sentencia;
BEGIN
    ...
    numero_tres_digitos := mi_numero_de_la_suerte;     --legal
    ...
    lo_he_encontrado := resultado_busqueda;               --legal
    ...
    sentencia_vacia := literal_nulo;                    --legal
    ...
END;
```

Autoevaluación

Indica la afirmación correcta.

○ Los subtipos lo único que hacen es añadir complejidad a nuestros programas.

○ No hay manera de restringir los subtipos con respecto a su tipo base.

○ Podemos definir un subtipo cuyo tipo base sea una tabla de la base de datos.

○ Podemos definir un subtipo de una variable pero no de una columna de la base de datos.

2.2.2.- Variables y constantes.

Llevamos un buen rato hablando de tipos de datos, variables e incluso de constantes y te estarás preguntando cuál es la forma adecuada de definirlas. En este apartado vamos a ver las diferentes posibilidades a la hora de definirlas y dejaremos para el apartado siguiente ver cuál es el lugar adecuado para hacerlo dentro de un bloque PL/SQL.

Para declarar variables o constantes pondremos el nombre de la variable, seguido del tipo de datos y opcionalmente una asignación. Si es una constante antepondremos la palabra **CONSTANT** al tipo de dato (lo que querrá decir que no podemos cambiar su valor). Podremos sustituir el operador de asignación en las declaraciones por la palabra reservada **DEFAULT**. También podremos forzar a que no sea nula utilizando la palabra **NOT NULL** después del tipo y antes de la asignación. Si restringimos una variable con **NOT NULL** deberemos asignarle un valor al declararla, de lo contrario PL/SQL lanzará la excepción **VALUE_ERROR** (no te asustes que más adelante veremos lo que son las excepciones, pero como adelanto te diré que es un error en tiempo de ejecución).

```
id SMALLINT;
hoy DATE := sysdate;
pi CONSTANT REAL:= 3.1415;
id SMALLINT NOT NULL;          --ilegal, no está inicializada
id SMALLINT NOT NULL := 9999;     --legal
```

El alcance y la visibilidad de las variables en PL/SQL será el típico de los lenguajes estructurados basados en bloques, aunque eso lo veremos más adelante.

Conversión de tipos.

Aunque en PL/SQL existe la conversión implícita de tipos para tipos parecidos, siempre es aconsejable utilizar la conversión explícita de tipos por medio de funciones de conversión (**TO_CHAR**, **TO_DATE**, **TO_NUMBER**, …) y así evitar resultados inesperados.

Precedencia de operadores.

Al igual que en nuestro lenguaje matemático se utiliza una precedencia entre operadores a la hora de realizar las operaciones aritméticas, en PL/SQL también se establece dicha precedencia para evitar confusiones. Si dos operadores tienen la misma precedencia lo aconsejable es utilizar los paréntesis (al igual que hacemos en nuestro lenguaje matemático) para alterar la precedencia de los mismos ya que las operaciones encerradas entre paréntesis tienen mayor precedencia. En la tabla siguiente se muestra la precedencia de los operadores de mayor a menor.

Precedencia de operadores.

Operador.	Operación.
`**`, `NOT`	Exponenciación, negación lógica.
`+`, `-`	Identidad, negación.
`*`, `/`	Multiplicación, división.
`+`, `-`, `||`	Suma, resta y concatenación.
`=`, `!=`, `<`, `>`, `<=`, `>=`, `IS NULL`, `LIKE`, `BETWEEN`, `IN`	Comparaciones.
`AND`	Conjunción lógica
`OR`	Disyunción lógica.

Autoevaluación

Rellena el hueco con el resultado de las siguientes operaciones.

- 5+3*2**2 es igual a: ☐.
- 2**3+6/3 es igual a: ☐.
- 2**(3+6/3) es igual a: ☐.

Enviar

2.3.- El bloque PL/SQL.

Ya hemos visto las unidades léxicas que componen PL/SQL, los tipos de datos que podemos utilizar y cómo se definen las variables y las constantes. Ahora vamos a ver la unidad básica en PL/SQL que es el bloque.

Un bloque PL/SQL consta de tres zonas:

- ✓ **Declaraciones**: definiciones de variables, constantes, cursores y excepciones.
- ✓ **Proceso**: zona donde se realizará el proceso en sí, conteniendo las sentencias ejecutables.
- ✓ **Excepciones**: zona de manejo de errores en tiempo de ejecución.

La sintaxis es la siguiente:

```
[DECLARE
     [Declaración de variables, constantes, cursores y excepciones]]
BEGIN
     [Sentencias ejecutables]
[EXCEPTION
     Manejadores de excepciones]
END;
```

Los bloques PL/SQL pueden anidarse a cualquier nivel. Como hemos comentado anteriormente el ámbito y la visibilidad de las variables es la normal en un lenguaje procedimental. Por ejemplo, en el siguiente fragmento de código se declara la variable `aux` en ambos bloques, pero en el bloque anidado `aux` con valor igual a 10 actúa de variable global y `aux` con valor igual a 5 actúa como variable local, por lo que en la comparación evaluaría a `FALSE`, ya que al tener el mismo nombre la visibilidad dominante sería la de la variable local.

```
DECLARE
     aux number := 10;
BEGIN
     DECLARE
          aux number := 5;
     BEGIN
          ...
          IF aux = 10 THEN     --evalúa a FALSE, no entraría
          ...
     END;
END;
```

Para saber más

En el siguiente enlace podrás ampliar información sobre el ámbito y la visibilidad de las variables en PL/SQL.

Ámbito y visibilidad en PL/SQL.

Autoevaluación

En PL/SQL el bloque es la unidad básica, por lo que éstos no pueden anidarse.

Verdadero.

○ Falso.

2.4.- Estructuras de control (I).

En la vida constantemente tenemos que tomar decisiones que hacen que llevemos a cabo unas acciones u otras dependiendo de unas circunstancias o repetir una serie de acciones un número dado de veces o hasta que se cumpla una condición. En PL/SQL también podemos imitar estas situaciones por medio de las estructuras de control que son sentencias que nos permiten manejar el flujo de control de nuestro programa, y éstas son dos: condicionales e iterativas.

Control condicional.

Las estructuras de control condicional nos permiten llevar a cabo una acción u otra dependiendo de una condición. Vemos sus diferentes variantes:

- ✓ **IF-THEN**: Forma más simple de las sentencias de control condicional. Si la evaluación de la condición es **TRUE**, entonces se ejecuta la secuencia de sentencias encerradas entre el **THEN** y el final de la sentencia.

Sentencia IF-THEN.

Sintaxis.

```
IF condición THEN
secuencia_de_sentencias;
END IF;
```

Ejemplo.

```
IF (b<>0) THEN
   c:=a/b;
END IF;
```

- ✓ **IF-THEN-ELSE**: Con esta forma de la sentencia ejecutaremos la primera secuencia de sentencias si la condición evalúa a **TRUE** y en caso contrario ejecutaremos la segunda secuencia de sentencias.

Sentencia IF-THEN-ELSE.

Sintaxis.

```
IF condición THEN
   Secuencia_de_sentencias1;
ELSE
   Secuencia_de_sentencias2;
END IF;
```

Ejemplo.

```
IF (b<>0) THEN
   c:=a/b;
END IF;
```

- ✓ **IF-THEN-ELSIF**: Con esta última forma de la sentencia condicional podemos hacer una selección múltiple. Si la evaluación de la condición 1 da **TRUE**, ejecutamos la secuencia de sentencias 1, sino evaluamos la condición 2. Si esta evalúa a **TRUE** ejecutamos la secuencia de sentencias 2 y así para todos los **ELSIF** que haya. El último **ELSE** es opcional y es por si no se cumple ninguna de las condiciones anteriores.

Sentencia IF-THEN-ELSIF.

Sintaxis.

```
IF condición_1 THEN
   Secuencia_de_sentencias1;
ELSIF condición_2 THEN
   Secuencia_de_sentencias2;
   ...
[ELSE
   Secuencia_de_sentencias;]
END IF;
```

Ejemplo.

```
IF (operación = 'SUMA') THEN
   resultado := arg1 + arg2;
ELSIF (operación = 'RESTA') THEN
   resultado := arg1 - arg2;
ELSIF (operación = 'PRODUCTO') THEN
   resultado := arg1 * arg2;
ELSIF (arg2 <> 0) AND (operación = 'DIVISIÓN') THEN
   resultado := arg1 / arg2;
ELSE
   RAISE operación_no_permitida;
END IF;
```

Autoevaluación

En PL/SQL no existen sentencias que nos permitan tomar una acción u otra dependiendo de una condición.

○ Verdadero.
○ Falso.

2.4.1.- Estructuras de control (II).

Ya que hemos visto las estructuras de control condicional, veamos ahora las estructuras de control iterativo.

Control iterativo.

Estas estructuras nos permiten ejecutar una secuencia de sentencias un determinado número de veces.

- ✓ **LOOP**: La forma más simple es el bucle infinito, cuya sintaxis es:

```
LOOP
   secuencia_de_sentencias;
END LOOP;
```

- ✓ **EXIT**: Con esta sentencia forzamos a un bucle a terminar y pasa el control a la siguiente sentencia después del bucle. Un **EXIT** no fuerza la salida de un bloque PL/SQL, sólo la salida del bucle.

```
LOOP
   …
   IF encontrado = TRUE THEN
      EXIT;
   END IF;
END LOOP;
```

- ✓ **EXIT WHEN condicion**: Fuerza a salir del bucle cuando se cumple una determinada condición.

```
LOOP
   …
   EXIT WHEN encontrado;
END LOOP;
```

- ✓ **WHILE LOOP**: Este tipo de bucle ejecuta la secuencia de sentencias mientras la condición sea cierta.

Sentencia WHILE LOOP.

Sintaxis.

```
WHILE condición LOOP
   Secuencia_de_sentencias;
END LOOP;
```

Ejemplo.

```
WHILE (not encontrado) LOOP
   …
END LOOP;
```

- ✓ **FOR LOOP**: Este bucle itera mientras el contador se encuentre en el rango definido.

Sentencia FOR LOOP.

Sintaxis.

```
FOR contador IN [REVERSE] limite_inferior..limite_superior LOOP
   Secuencia_de_sentencias;
END LOOP;
```

Ejemplo.

```
FOR i IN 1..3 LOOP    --i=1, i=2, i=3
   …
END LOOP;

SELECT count(*) INTO num_agentes FROM agentes;
FOR i IN 1..num_agentes LOOP
   …
END LOOP;


FOR i IN REVERSE 1..3 LOOP --i=3, i=2, i=1
   …
END LOOP;
```

Autoevaluación

Al utilizar REVERSE en un bucle FOR, en el rango debemos poner el número mayor el primero y el menor el último.

○ Verdadero.
○ Falso.

2.5.- Manejo de errores (I).

Muchas veces te habrá pasado que surgen situaciones inesperadas con las que no contabas y a las que tienes que hacer frente. Pues cuando programamos con PL/SQL pasa lo mismo, que a veces tenemos que manejar errores debidos a situaciones diversas. Vamos a ver cómo tratarlos.

Cualquier situación de error es llamada **excepción** en PL/SQL. Cuando se detecta un error, una excepción es lanzada, es decir, la ejecución normal se para y el control se transfiere a la parte de manejo de excepciones. La parte de manejo de excepciones es la parte etiquetada como EXCEPTION y constará de sentencias para el manejo de dichas excepciones, llamadas **manejadores de excepciones**.

Manejadores de excepciones.

Sintaxis.

```
WHEN nombre_excepcion THEN

     ....
WHEN OTHERS THEN
```

Ejemplo.

```
DECLARE
     supervisor agentes%ROWTYPE;
BEGIN
     SELECT * INTO supervisor FROM agentes
     WHERE categoria = 2 AND oficina = 3;
          ...
EXCEPTION
     WHEN NO_DATA_FOUND THEN
     --Manejamos el no haber encontrado datos
WHEN OTHERS THEN
     --Manejamos cualquier error inesperado
END;
```

La parte OTHERS captura cualquier excepción no capturada.

Las excepciones pueden estar definidas por el usuario o definidas internamente. Las excepciones predefinidas se lanzarán automáticamente asociadas a un error de Oracle. Las excepciones definidas por el usuario deberán definirse y lanzarse explícitamente.

En PL/SQL nosotros podemos definir nuestras propias excepciones en la parte DECLARE de cualquier bloque. Estas excepciones podemos lanzarlas explícitamente por medio de la sentencia RAISE nombre_excepción.

Excepciones definidas por el usuario.

Sintaxis.

```
DECLARE
     nombre_excepcion EXCEPTION;
BEGIN
     ...
     RAISE nombre_excepcion;
     ...
END;
```

Ejemplo.

```
DECLARE
     categoria_erronea EXCEPTION;
BEGIN
     ...
     IF categoria<0 OR categoria>3 THEN
          RAISE categoria_erronea;
     END IF;
     ...
EXCEPTION
     WHEN categoria_erronea THEN
          --manejamos la categoria errónea
END;
```

Debes conocer

En el siguiente enlace podrás ver las diferentes excepciones predefinidas en Oracle, junto a su código de error asociado (que luego veremos lo que es) y una explicación de cuándo son lanzadas.

Excepciones predefinidas en Oracle.

2.5.1.- Manejo de errores (II).

Ahora que ya sabemos lo que son las excepciones, cómo capturarlas y manejarlas y cómo definir y lanzar las nuestras propias. Es la hora de comentar algunos detalles sobre el uso de las mismas.

- ✓ El alcance de una excepción sigue las mismas reglas que el de una variable, por lo que si nosotros redefinimos una excepción que ya es global para el bloque, la definición local prevalecerá y no podremos capturar esa excepción a menos que el bloque en la que estaba definida esa excepción fuese un bloque nombrado, y podremos capturarla usando la sintaxis: **nombre_bloque.nombre_excepcion**.
- ✓ Las excepciones predefinidas están definidas globalmente. No necesitamos (ni debemos) redefinir las excepciones predefinidas.

```
DECLARE
    no_data_found EXCEPTION;
BEGIN
    SELECT * INTO ...
EXCEPTION
    WHEN no_data_found THEN     --captura la excepción local, no
                    --la global
END;
```

- ✓ Cuando manejamos una excepción no podemos continuar por la siguiente sentencia a la que la lanzó.

```
DECLARE
    ...
BEGIN
    ...
    INSERT INTO familias VALUES
(id_fam, nom_fam, NULL, oficina);
    INSERT INTO agentes VALUES
(id_ag, nom_ag, login, password, 0, 0, id_fam, NULL);
    ...
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        --manejamos la excepción debida a que el nombre de
        --la familia ya existe, pero no podemos continuar por
        --el INSERT INTO agentes, a no ser que lo pongamos
        --explícitamente en el manejador
END;
```

- ✓ Pero sí podemos encerrar la sentencia dentro de un bloque, y ahí capturar las posibles excepciones, para continuar con las siguientes sentencias.

```
DECLARE
    id_fam NUMBER;
    nom_fam VARCHAR2(40);
    oficina NUMBER;
    id_ag NUMBER;
    nom_ag VARCHAR2(60);
    usuario VARCHAR2(20);
    clave VARCHAR2(20);
BEGIN
    ...
    BEGIN
        INSERT INTO familias VALUES (id_fam, nom_fam, NULL, oficina);
    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
SELECT identificador INTO id_fam FROM familias WHERE nombre = nom_fam;
    END;
    INSERT INTO agentes VALUES (id_ag, nom_ag, login, password, 1, 1, id_fam, null);
    ...
END;
```

Ejercicio resuelto

Supongamos que queremos reintentar una transacción hasta que no nos dé ningún error. Para ello deberemos encapsular la transacción en un bloque y capturar en éste las posibles excepciones. El bloque lo metemos en un bucle y así se reintentará la transacción hasta que sea posible llevarla a cabo.

2.5.2.- Manejo de errores (III).

Continuemos viendo algunos detalles a tener en cuenta, relativos al uso de las excepciones.

- Cuando ejecutamos varias sentencias seguidas del mismo tipo y queremos capturar alguna posible excepción debida al tipo de sentencia, podemos encapsular cada sentencia en un bloque y manejar en cada bloque la excepción, o podemos utilizar una variable localizadora para saber qué sentencia ha sido la que ha lanzado la excepción (aunque de esta manera no podremos continuar por la siguiente sentencia).

```
DECLARE
    sentencia NUMBER := 0;
BEGIN
    ...
    SELECT * FROM agentes ...
    sentencia := 1;
    SELECT * FROM familias ...
    sentencia := 2;
    SELECT * FROM oficinas ...
    ...
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        IF sentencia = 0 THEN
            RAISE agente_no_encontrado;
        ELSIF sentencia = 1 THEN
            RAISE familia_no_encontrada;
        ELSIF sentencia = 2 THEN
            RAISE oficina_no_encontrada;
        END IF;
END;
```

- Si la excepción es capturada por un manejador de excepción apropiado, ésta es tratada y posteriormente el control es devuelto al bloque superior. Si la excepción no es capturada y no existe bloque superior, el control se devolverá al entorno. También puede darse que la excepción sea manejada en un bloque superior a falta de manejadores para ella en los bloques internos, la excepción se propaga de un bloque al superior y así hasta que sea manejada o no queden bloques superiores con lo que el control se devuelve al entorno. Una excepción también puede ser relanzada en un manejador. En la siguiente presentación puedes ver cómo se propagan diferentes excepciones entre diferentes bloques.

Resumen textual alternativo

Autoevaluación

Todas las excepciones están predefinidas y nosotros no podemos definir nuevas excepciones.

○ Verdadero.
○ Falso.

Las excepciones definidas por el usuario deben ser lanzadas explícitamente.

○ Verdadero.
○ Falso.

Es obligatorio declarar todas las excepciones predefinidas que vamos a usar en nuestros bloques.

○ Verdadero.
○ Falso.

2.5.3.- Manejo de errores (IV).

Oracle también permite que nosotros lancemos nuestros propios mensajes de error a las aplicaciones y asociarlos a un código de error que Oracle reserva, para no interferir con los demás códigos de error. Lo hacemos por medio del procedimiento:

```
RAISE_APPLICATION_ERROR(error_number, message [, {TRUE|FALSE}]);
```

Donde **error_number** es un entero negativo comprendido entre –20000..-20999 y **message** es una cadena que devolvemos a la aplicación. El tercer parámetro especifica si el error se coloca en la pila de errores (**TRUE**) o se vacía la pila y se coloca únicamente el nuestro (**FALSE**). Sólo podemos llamar a este procedimiento desde un subprograma.

No hay excepciones predefinidas asociadas a todos los posibles errores de Oracle, por lo que nosotros podremos asociar excepciones definidas por nosotros a errores Oracle, por medio de la directiva al compilador (o pseudoinstrucción):

```
PRAGMA_INIT( nombre_excepcion, error_Oracle )
```

Donde **nombre_excepcion** es el nombre de una excepción definida anteriormente, y **error_Oracle** es el número negativo asociado al error.

```
DECLARE
    no_null EXCEPTION;
    PRAGMA EXCEPTION_INIT(no_null, -1400);
    id familias.identificador%TYPE;
    nombre familias.nombre%TYPE;
BEGIN
    ...
    nombre := NULL;
    ...
    INSERT INTO familias VALUES (id, nombre, null, null);
EXCEPTION
    WHEN no_null THEN
        ...
END;
```

Oracle asocia 2 funciones para comprobar la ejecución de cualquier sentencia. **SQLCODE** nos devuelve el código de error y **SQLERRM** devuelve el mensaje de error asociado. Si una sentencia es ejecutada correctamente, **SQLCODE** nos devuelve 0 y en caso contrario devolverá un número negativo asociado al error (excepto **NO_DATA_FOUND** que tiene asociado el +100).

```
DECLARE
cod number;
msg varchar2(100);
BEGIN
...
EXCEPTION
WHEN OTHERS THEN
    cod := SQLCODE;
    msg := SUBSTR(SQLERRM, 1, 100);
    INSERT INTO errores VALUES (cod, msg);
END;
```

Autoevaluación

De las siguientes afirmaciones marca las que creas que son correctas.

☐ Podemos lanzar nuestros propios mensajes de error a las aplicaciones.
☐ Podemos acceder al código de error generado por la ejecución de una sentencia pero no a su mensaje asociado.
☐ Podemos asociar excepciones definidas por nosotros a códigos de error de Oracle.

Mostrar Información

3.- Tipos de datos compuestos.

Caso práctico

María, gracias a la ayuda de Juan, ha comprendido muy bien el manejo básico de PL/SQL. **Juan** le comenta que en la mayoría de los lenguajes de programación, además de los tipos de datos simples, existen tipos de datos compuestos que le dan mucha más versatilidad a los mismos y una gran potencia. **Juan** no conoce bien si PL/SQL dispone de estos tipos de datos y si dispone de ellos ¿de cuáles?

María y **Juan** van a hablar con **Ada** para ver si les puede aclarar un poco la situación y dar unas pequeñas pautas para empezar. **Ada** les comenta que claro que PL/SQL cuenta con este tipo de datos, pero que hoy tiene una reunión importantísima y que tiene que terminar de preparársela, por lo que los remite al capítulo sobre tipos de datos compuestos del manual que les pasó. **Juan** y **María** le dicen que no se preocupe y que ya verá como a la vuelta de esa reunión son capaces de saber cuáles son e incluso de dominarlos. **Ada**, para motivarlos, les dice que si es así los invita a un aperitivo a su vuelta. Así que **Juan** y **María** se ponen con el capítulo de tipos de datos compuestos del manual para ver si pueden sorprender a **Ada** a su vuelta.

En el capítulo anterior, entre otras cosas, hemos conocido los tipos de datos simples con los que cuenta PL/SQL. Pero dependiendo de la complejidad de los problemas, necesitamos disponer de otras estructuras en las que apoyarnos para poder modelar nuestro problema. En este capítulo nos vamos a centrar en conocer los tipos de datos complejos que nos ofrece PL/SQL y cómo utilizarlos para así poder sacarle el mayor partido posible.

Citas para pensar

"Todo lo complejo puede dividirse en partes simples".

René Descartes.

3.1.- Registros.

El uso de los registros es algo muy común en los lenguajes de programación. PL/SQL también nos ofrece este tipo de datos. En este apartado veremos qué son y cómo definirlos y utilizarlos.

> Un registro es un grupo de elementos relacionados almacenados en campos, cada uno de los cuales tiene su propio nombre y tipo de dato.

Por ejemplo, una dirección podría ser un registro con campos como calle, número, piso, puerta, código postal, ciudad, provincia y país. Los registros hacen que la información sea más fácil de organizar y representar. Para declarar un registro seguiremos la siguiente sintaxis:

```
TYPE nombre_tipo IS RECORD (decl_campo[, decl_campo] ...);
```

donde:

```
decl_campo := nombre tipo [[NOT NULL] {:=|DEFAULT} expresion]
```

El tipo del campo será cualquier tipo de dato válido en PL/SQL excepto **REF CURSOR**. La expresión será cualquier expresión que evalúe al tipo de dato del campo.

```
TYPE direccion IS RECORD
(
calle          VARCHAR2(50),
numero          INTEGER(4),
piso          INTEGER(4),
puerta          VARCHAR2(2),
codigo_postal     INTEGER(5),
ciudad          VARCHAR2(30),
provincia     VARCHAR2(20),
pais          VARCHAR2(20) := 'España'
);
mi_direccion direccion;
```

Para acceder a los campos usaremos la notación del punto.

```
...
mi_direccion.calle := 'Ramirez Arellano';
mi_direccion.numero := 15;
...
```

Para asignar un registro a otro, éstos deben ser del mismo tipo, no basta que tengan el mismo número de campos y éstos emparejen uno a uno. Tampoco podemos comparar registros aunque sean del mismo tipo, ni tampoco comprobar si éstos son nulos. Podemos hacer **SELECT** en registros, pero no podemos hacer **INSERT** desde registros.

```
DECLARE
TYPE familia IS RECORD
(
    identificador     NUMBER,
    nombre          VARCHAR2(40),
    padre          NUMBER,
    oficina          NUMBER
);
TYPE familia_aux IS RECORD
(
    identificador     NUMBER,
    nombre          VARCHAR2(40),
    padre          NUMBER,
    oficina          NUMBER
    );
    SUBTYPE familia_fila IS familias%ROWTYPE;
    mi_fam familia;
    mi_fam_aux familia_aux;
    mi_fam_fila familia_fila;
BEGIN
...
mi_fam := mi_fam_aux;                          --ilegal
mi_fam := mi_fam_fila;                          --legal
IF mi_fam IS NULL THEN ...                     --ilegal
IF mi_fam = mi_fam_fila THEN ...                --ilegal
SELECT * INTO mi_fam FROM familias ...          --legal
INSERT INTO familias VALUES (mi_fam_fila);          --ilegal
...
END;
```

Autoevaluación

Un registro se puede asignar a otro siempre que tenga el mismo número de campos y éstos emparejen uno a uno.

○ Verdadero.
○ Falso.

3.2.- Colecciones. Arrays de longitud variable.

Una colección es un grupo ordenado de elementos, todos del mismo tipo. Cada elemento tiene un subíndice único que determina su posición en la colección.

En PL/SQL las colecciones sólo pueden tener una dimensión. PL/SQL ofrece 2 clases de colecciones: arrays de longitud variable y tablas anidadas.

Arrays de longitud variable.

Los elementos del tipo **VARRAY** son los llamados arrays de longitud variable. Son como los arrays de cualquier otro lenguaje de programación, pero con la salvedad de que a la hora de declararlos, nosotros indicamos su tamaño máximo y el array podrá ir creciendo dinámicamente hasta alcanzar ese tamaño. Un **VARRAY** siempre tiene un límite inferior igual a 1 y un límite superior igual al tamaño máximo.

Para declarar un **VARRAY** usaremos la sintaxis:

```
TYPE nombre IS {VARRAY | VARYING} (tamaño_máximo) OF tipo_elementos [NOT NULL];
```

Donde **tamaño_máximo** será un entero positivo y **tipo_elementos** será cualquier tipo de dato válido en PL/SQL, excepto **BINARY_INTEGER, BOOLEAN, LONG, LONG RAW, NATURAL, NATURALN, NCHAR, NCLOB, NVARCHAR2**, objetos que tengan como atributos **TABLE** o **VARRAY**, **PLS_INTEGER, POSITIVE, POSITIVEN, SIGNTYPE, STRING, TABLE, VARRAY**. Si **tipo_elementos** es un registro, todos los campos deberían ser de un tipo escalar.

Cuando definimos un **VARRAY**, éste es automáticamente nulo, por lo que para empezar a utilizarlo deberemos inicializarlo. Para ello podemos usar un constructor:

```
TYPE familias_hijas IS VARRAY(100) OF familia;
familias_hijas1 familias_hijas := familias_hijas( familia(100, 'Fam100', 10, null), ..., familia(105, 'Fam105', 10, null));
```

También podemos usar constructores vacíos.

```
familias_hijas2 familias_hijas := familias_hijas();
```

Para referenciar elementos en un **VARRAY** utilizaremos la sintaxis **nombre_colección(subíndice)**. Si una función devuelve un **VARRAY**, podemos usar la sintaxis: **nombre_funcion(lista_parametros)(subindice)**.

```
IF familias_hijas1(1).identificador = 100 THEN ...
IF dame_familias_hijas(10)(1).identificador = 100 THEN ...
```

Un **VARRAY** puede ser asignado a otro si ambos son del mismo tipo.

```
DECLARE
TYPE tabla1 IS VARRAY(10) OF NUMBER;
TYPE tabla2 IS VARRAY(10) OF NUMBER;
mi_tabla1 tabla1 := tabla1();
mi_tabla2 tabla2 := tabla2();
mi_tabla tabla1 := tabla1();
BEGIN
...
mi_tabla := mi_tabla1;          --legal
mi_tabla1 := mi_tabla2;     --ilegal
...
END;
```

Para extender un **VARRAY** usaremos el método **EXTEND**. Sin parámetros, extendemos en 1 elemento nulo el **VARRAY**. **EXTEND(n)** añade n elementos nulos al **VARRAY** y **EXTEND(n,i)** añade n copias del i-ésimo elemento.

COUNT nos dirá el número de elementos del **VARRAY**. **LIMIT** nos dice el tamaño máximo del **VARRAY**. **FIRST** siempre será 1. **LAST** siempre será igual a **COUNT**. **PRIOR** y **NEXT** devolverá el antecesor y el sucesor del elemento.

Al trabajar con **VARRAY** podemos hacer que salte alguna de las siguientes excepciones, debidas a un mal uso de los mismos: **COLECTION_IS_NULL, SUBSCRIPT_BEYOND_COUNT, SUBSCRIPT_OUTSIDE_LIMIT** y **VALUE_ERROR**.

Ejemplos de uso de los VARRAY.

Extender un VARRAY.	Consultar propiedades VARRAY.	Posibles excepciones.

Extender un VARRAY.

```
DECLARE
   TYPE tab_num IS VARRAY(10) OF NUMBER;
   mi_tab tab_num;
BEGIN
   mi_tab := tab_num();
   FOR i IN 1..10 LOOP
      mi_tab.EXTEND;
      mi_tab(i) := calcular_elemento(i);
   END LOOP;
   ...
END;
```

Consultar propiedades VARRAY.

```
DECLARE
   TYPE numeros IS VARRAY(20) OF NUMBER;
   tabla_numeros numeros := numeros();
   num NUMBER;
BEGIN
   num := tabla_numeros.COUNT;          --num := 0
   FOR i IN 1..10 LOOP
      tabla_numeros.EXTEND;
      tabla_numeros(i) := i;
   END LOOP;
   num := tabla_numeros.COUNT; --num := 10
   num := tabla_numeros.LIMIT; --num := 20
   num := tabla_numeros.FIRST; --num := 1;
   num := tabla_numeros.LAST; --num := 10;
   ...
END;
```

Posibles excepciones.

```
DECLARE
  TYPE numeros IS VARRAY(20) OF INTEGER;
   v_numeros numeros := numeros( 10, 20, 30, 40 );
    v_enteros numeros;
BEGIN
    v_enteros(1) := 15; --lanzaría COLECTION_IS_NULL
    v_numeros(5) := 20; --lanzaría SUBSCRIPT_BEYOND_COUNT
    v_numeros(-1) := 5; --lanzaría SUBSCRIPT_OUTSIDE_LIMIT
   v_numeros('A') := 25; --lanzaría VALUE_ERROR
    ....
END;
```

Autoevaluación

Indica, de entre las siguientes, cuál es la afirmación correcta referida a VARRAY.

○ Un VARRAY no hace falta inicializarlo.
○ COUNT y LIMIT siempre nos devolverán el mismo valor.
○ LAST y COUNT siempre nos devolverán el mismo valor.

3.2.1.- Colecciones. Tablas anidadas.

Las tablas anidadas son colecciones de elementos, que no tienen límite superior fijo, y pueden aumentar dinámicamente su tamaño. Además podemos borrar elementos individuales.

Para declararlos utilizaremos la siguiente sintaxis:

```
TYPE nombre IS TABLE OF tipo_elementos [NOT NULL];
```

Donde **tipo_elementos** tendrá las mismas restricciones que para los **VARRAY**.

Al igual que pasaba con los **VARRAY**, al declarar una tabla anidada, ésta es automáticamente nula, por lo que deberemos inicializarla antes de usarla.

```
TYPE hijos IS TABLE OF agente;
hijos_fam hijos := hijos( agente(...) ...);
```

También podemos usar un constructor nulo.

Para referenciar elementos usamos la misma sintaxis que para los **VARRAY**.

Para extender una tabla usamos **EXTEND** exactamente igual que para los **VARRAY**. **COUNT** nos dirá el número de elementos, que no tiene por qué coincidir con **LAST**. **LIMIT** no tiene sentido y devuelve **NULL**. **EXISTS(n)** devuelve **TRUE** si el elemento existe, y **FALSE** en otro caso (el elemento ha sido borrado). **FIRST** devuelve el primer elemento que no siempre será 1, ya que hemos podido borrar elementos del principio. **LAST** devuelve el último elemento. **PRIOR** y **NEXT** nos dicen el antecesor y sucesor del elemento (ignorando elementos borrados). **TRIM** sin argumentos borra un elemento del final de la tabla. **TRIM(n)** borra n elementos del final de la tabla. **TRIM** opera en el tamaño interno, por lo que si encuentra un elemento borrado con **DELETE**, lo incluye para ser eliminado de la colección. DELETE(n) borra el n-ésimo elemento. **DELETE(n, m)** borra del elemento n al m. Si después de hacer **DELETE**, consultamos si el elemento existe nos devolverá **FALSE**.

Al trabajar con tablas anidadas podemos hacer que salte alguna de las siguientes excepciones, debidas a un mal uso de las mismas: **COLECTION_IS_NULL**, **NO_DATA_FOUND**, **SUBSCRIPT_BEYOND_COUNT** y **VALUE_ERROR**.

Ejemplos de uso de las tablas anidadas.

Diferentes operaciones sobre tablas anidadas.

```
DECLARE
TYPE numeros IS TABLE OF NUMBER;
tabla_numeros numeros := numeros();
num NUMBER;
BEGIN
num := tabla_numeros.COUNT;     --num := 0
FOR i IN 1..10 LOOP
     tabla_numeros.EXTEND;
     tabla_numeros(i) := i;
END LOOP;
num := tabla_numeros.COUNT;     --num := 10
tabla_numeros.DELETE(10);
num := tabla_numeros.LAST;     --num := 9
num := tabla_numeros.FIRST;     --num := 1
tabla_numeros.DELETE(1);
num := tabla_numeros.FIRST;     --num := 2
FOR i IN 1..4 LOOP
     tabla_numeros.DELETE(2*i);
END LOOP;
num := tabla_numeros.COUNT;     --num := 4
num := tabla_numeros.LAST;     --num := 9
...
END;
```

Posibles excepciones en su uso.

```
DECLARE
TYPE numeros IS TABLE OF NUMBER;
tabla_num numeros := numeros();
tabla1 numeros;
BEGIN
tabla1(5) := 0;     --lanzaría COLECTION_IS_NULL
tabla_num.EXTEND(5);
tabla_num.DELETE(4);
tabla_num(4) := 3;     --lanzaría NO_DATA_FOUND
tabla_num(6) := 10;     --lanzaría SUBSCRIPT_BEYOND_COUNT
tabla_num(-1) := 0;     --lanzaría SUBSCRIPT_OUTSIDE_LIMIT
tabla_num(‘y’) := 5;--lanzaría VALUE_ERROR
END;
```

Autoevaluación

Las tablas anidadas podemos hacer que crezcan dinámicamente, pero no podemos borrar elementos.

Verdadero.

○ Falso.

3.3.- Cursores.

En los apartados anteriores hemos visto algunos tipos de datos compuestos cuyo uso es común en otros lenguajes de programación. Sin embargo, en este apartado vamos a ver un tipo de dato, que aunque se puede asemejar a otros que ya conozcas, su uso es exclusivo en la programación de las bases de datos y que es el **cursor**.

> Un cursor no es más que una estructura que almacena el conjunto de filas devuelto por una consulta a la base de datos.

Oracle usa áreas de trabajo para ejecutar sentencias SQL y almacenar la información procesada. Hay 2 clases de cursores: **implícitos** y **explícitos**. PL/SQL declara implícitamente un cursor para todas las sentencias SQL de manipulación de datos, incluyendo consultas que devuelven una sola fila. Para las consultas que devuelven más de una fila, se debe declarar explícitamente un cursor para procesar las filas individualmente.

En este primer apartado vamos a hablar de los cursores implícitos y de los atributos de un cursor (estos atributos tienen sentido con los cursores explícitos, pero los introducimos aquí para ir abriendo boca), para luego pasar a ver los cursores explícitos y terminaremos hablando de los cursores variables.

Cursores implícitos.

Oracle abre implícitamente un cursor para procesar cada sentencia SQL que no esté asociada con un cursor declarado explícitamente.

Con un cursor implícito no podemos usar las sentencias **OPEN**, **FETCH** y **CLOSE** para controlar el cursor. Pero sí podemos usar los atributos del cursor para obtener información sobre las sentencias SQL más recientemente ejecutadas.

Atributos de un cursor.

Cada cursor tiene 4 atributos que podemos usar para obtener información sobre la ejecución del mismo o sobre los datos. Estos atributos pueden ser usados en PL/SQL, pero no en SQL. Aunque estos atributos se refieren en general a cursores explícitos y tienen que ver con las operaciones que hayamos realizado con el cursor, es deseable comentarlas aquí y en el siguiente apartado tomarán pleno sentido.

- ✓ **%FOUND**: Después de que el cursor esté abierto y antes del primer **FETCH**, **%FOUND** devuelve **NULL**. Después del primer **FETCH**, **%FOUND** devolverá **TRUE** si el último **FETCH** ha devuelto una fila y **FALSE** en caso contrario. Para cursores implícitos **%FOUND** devuelve **TRUE** si un **INSERT**, **UPDATE** o **DELETE** afectan a una o más de una fila, o un **SELECT ... INTO ...** devuelve una o más filas. En otro caso **%FOUND** devuelve **FALSE**.
- ✓ **%NOTFOUND**: Es lógicamente lo contrario a **%FOUND**.
- ✓ **%ISOPEN**: Evalúa a **TRUE** si el cursor está abierto y **FALSE** en caso contrario. Para cursores implícitos, como Oracle los cierra automáticamente, **%ISOPEN** evalúa siempre a **FALSE**.
- ✓ **%ROWCOUNT**: Para un cursor abierto y antes del primer **FETCH**, **%ROWCOUNT** evalúa a 0. Después de cada **FETCH**, **%ROWCOUNT** es incrementado y evalúa al número de filas que hemos procesado. Para cursores implícitos **%ROWCOUNT** evalúa al número de filas afectadas por un **INSERT**, **UPDATE** o **DELETE** o el número de filas devueltas por un **SELECT ... INTO ...**

Debes conocer

Aunque todavía no hemos visto las operaciones que se pueden realizar con un cursor explícito, es conveniente que te vayas familiarizando con la evaluación de sus atributos según las operaciones que hayamos realizado con el cursor y que tomarán pleno sentido cuando veamos el siguiente apartado.

Evaluación de los atributos de un cursor explícito.

3.3.1.- Cursores explícitos.

Cuando una consulta devuelve múltiples filas, podemos declarar explícitamente un cursor para procesar las filas devueltas. Cuando declaramos un cursor, lo que hacemos es darle un nombre y asociarle una consulta usando la siguiente sintaxis:

```
CURSOR nombre_cursor [(parametro [, parametro] ...)] [RETURN tipo_devuelto] IS sentencia_select;
```

Donde **tipo_devuelto** debe representar un registro o una fila de una tabla de la base de datos, y **parámetro** sigue la siguiente sintaxis:

```
parametro := nombre_parametro [IN] tipo_dato [{:= | DEFAULT} expresion]
```

Ejemplos:

```
CURSOR cAgentes IS SELECT * FROM agentes;
CURSOR cFamilias RETURN familias%ROWTYPE IS SELECT * FROM familias WHERE ...
```

Además, como hemos visto en la declaración, un cursor puede tomar parámetros, los cuales pueden aparecer en la consulta asociada como si fuesen constantes. Los parámetros serán de entrada, un cursor no puede devolver valores en los parámetros actuales. A un parámetro de un cursor no podemos imponerle la restricción **NOT NULL**.

```
CURSOR c1 (cat INTEGER DEFAULT 0) IS SELECT * FROM agentes WHERE categoria = cat;
```

Cuando abrimos un cursor, lo que se hace es ejecutar la consulta asociada e identificar el conjunto resultado, que serán todas las filas que emparejen con el criterio de búsqueda de la consulta. Para abrir un cursor usamos la sintaxis:

```
OPEN nombre_cursor [(parametro [, parametro] ...)];
```

Ejemplos:

```
OPEN cAgentes;
OPEN c1(1);
OPEN c1;
```

La sentencia **FETCH** devuelve una fila del conjunto resultado. Después de cada **FETCH**, el cursor avanza a la próxima fila en el conjunto resultado.

```
FETCH cFamilias INTO mi_id, mi_nom, mi_fam, mi_ofi;
```

Para cada valor de columna devuelto por la consulta asociada al cursor, debe haber una variable que se corresponda en la lista de variables después del **INTO**.

Para procesar un cursor entero deberemos hacerlo por medio de un bucle.

```
BEGIN
...
OPEN cFamilias;
LOOP
    FETCH cFamilias INTO mi_id, mi_nom, mi_fam, mi_ofi;
    EXIT WHEN cFamilias%NOTFOUND;
    ...
END LOOP;
...
END;
```

Una vez procesado el cursor, deberemos cerrarlo, con lo que deshabilitamos el cursor y el conjunto resultado queda indefinido.

```
CLOSE cFamilias;
```

Una vez cerrado el cursor podemos reabrirlo, pero cualquier otra operación que hagamos con el cursor cerrado lanzará la excepción **INVALID_CURSOR**.

También podemos simplificar la operación de procesamiento de un cursor, por medio de los bucles para cursores, los cuales declaran implícitamente una variable índice definida como **%ROWTYPE** para el cursor, abren el cursor, se van trayendo los valores de cada fila del cursor, almacenándolas en la variable índice, y finalmente cierran el cursor.

```
BEGIN
    ...
    FOR cFamilias_rec IN cFamilias LOOP
        --Procesamos las filas accediendo a
        --cFamilias_rec.identificador, cFamilias_rec.nombre,
        --cFamilias_rec.familia, ...
    END LOOP;
    ...
END;
```

Autoevaluación

En PL/SQL los cursores son abiertos al definirlos.

○ Verdadero.

○ Falso.

3.3.2.- Cursores variables.

Oracle, además de los cursores vistos anteriormente, nos permite definir cursores variables que son como punteros a cursores, por lo que podemos usarlos para referirnos a cualquier tipo de consulta. Los cursores serían estáticos y los cursores variables serían dinámicos.

Para declarar un cursor variable debemos seguir 2 pasos:

- ✓ Definir un tipo **REF CURSOR** y entonces declarar una variable de ese tipo.

```
TYPE tipo_cursor IS REF CURSOR RETURN agentes%ROWTYPE;
cAgentes tipo_cursor;
```

- ✓ Una vez definido el cursor variable debemos asociarlo a una consulta (notar que esto no se hace en la parte declarativa, sino dinámicamente en la parte de ejecución) y esto lo hacemos con la sentencia **OPEN-FOR** utilizando la siguiente sintaxis:

```
OPEN nombre_variable_cursor FOR sentencia_select;

OPEN cAgentes FOR SELECT * FROM agentes WHERE oficina = 1;
```

Un cursor variable no puede tomar parámetros. Podemos usar los atributos de los cursores para cursores variables.

Además, podemos usar varios **OPEN-FOR** para abrir el mismo cursor variable para diferentes consultas. No necesitamos cerrarlo antes de reabrirlo. Cuando abrimos un cursor variable para una consulta diferente, la consulta previa se pierde.

Una vez abierto el cursor variable, su manejo es idéntico a un cursor. Usaremos **FETCH** para traernos las filas, usaremos sus atributos para hacer comprobaciones y lo cerraremos cuando dejemos de usarlo.

```
DECLARE
TYPE cursor_Agentes IS REF CURSOR RETURN agentes%ROWTYPE;
cAgentes cursor_Agentes;
agente cAgentes%ROWTYPE;
BEGIN
...
OPEN cAgentes FOR SELECT * FROM agentes WHERE oficina = 1;
LOOP
    FETCH cAgentes INTO agente;
    EXIT WHEN cAgentes%NOTFOUND;
    ...
END LOOP;
CLOSE cAgentes;
...
END;
```

Autoevaluación

A los cursores variables no podemos pasarles parámetros al abrirlos.

○ Verdadero.
○ Falso.

Los cursores variables se abren exactamente igual que los cursores explícitos.

○ Verdadero.
○ Falso.

4.- Abstracción en PL/SQL.

Caso práctico

María, gracias a la ayuda de **Juan**, tiene bastante claro cómo programar en PL/SQL pero no entiende muy bien cómo integrar todo esto con la base de datos de juegos on-line. Sabe que puede utilizar unos tipos de datos, que hay unas estructuras de control y que se pueden manejar los errores que surjan, pero lo que no sabe es cómo y donde utilizar todo eso.

Juan le explica que lo que hasta ahora ha aprendido es el comienzo, pero que ahora viene lo bueno y que será donde le va a encontrar pleno sentido a lo aprendido anteriormente. Le explica que PL/SQL permite crear funciones y procedimientos y además agruparlos en paquetes y que eso será lo que realmente van a hacer con la base de datos de juegos on-line. Deberán ver qué es lo que utilizan más comúnmente e implementarlo en PL/SQL utilizando funciones y procedimientos según convenga. **Juan** la tranquiliza y le dice que lo primero que va a hacer es explicarle cómo se escriben dichas funciones y procedimientos y luego pasarán a implementar alguno y que así verá la potencia real de PL/SQL. **María** se queda más tranquila y está deseando implementar esa primera función o procedimiento que le resolverá la gran duda que tiene.

Hoy día cualquier lenguaje de programación permite definir diferentes grados de abstracción en sus programas. La abstracción permite a los programadores crear unidades lógicas y posteriormente utilizarlas pensando en qué hace y no en cómo lo hace. La abstracción se consigue utilizando funciones, procedimientos, librerías, objetos, etc.

PL/SQL nos permite definir funciones y procedimientos. Además nos permite agrupar todas aquellas que tengan relación en paquetes. También permite la utilización de objetos. Todo esto es lo que veremos en este apartado y conseguiremos darle modularidad a nuestras aplicaciones, aumentar la reusabilidad y mantenimiento del código y añadir grados de abstracción a los problemas.

Para saber más

En los siguientes enlaces puedes ampliar información sobre la abstracción en programación.

Abstracción en los lenguajes de programación.

Programación dirigida a objetos.

4.1.- Subprogramas.

Los subprogramas son bloques nombrados a los cuales les podemos pasar parámetros y los podemos invocar. Además, los subprogramas pueden estar almacenados en la base de datos o estar encerrados en otros bloques. Si el programa está almacenado en la base de datos, podremos invocarlo si tenemos permisos suficientes y si está encerrado en otro bloque lo podremos invocar si tenemos visibilidad sobre el mismo.

Hay dos clases de subprogramas: las funciones y los procedimientos. Las funciones devuelven un valor y los procedimientos no.

Para declarar un subprograma utilizamos la siguiente sintaxis:

Sintaxis para la declaración de subprogramas.

Funciones.

```
FUNCTION nombre [(parametro [, parametro] ...)]
RETURN tipo_dato IS
[declaraciones_locales]
BEGIN
sentencias_ejecutables
[EXCEPTION
manejadores_de_excepciones]
END [nombre];
```

Procedimientos.

```
PROCEDURE nombre [( parametro [, parametro] ... )] IS
    [declaraciones_locales]
BEGIN
sentencias_ejecutables
[EXCEPTION
manejadores_de_excepciones]
END [nombre];
```

Donde:

```
parametro := nombre_parametro [IN|OUT|IN OUT] tipo_dato [{:=|DEFAULT} expresion]
```

Algunas consideraciones que debes tener en cuenta son las siguientes:

- ✓ No podemos imponer una restricción **NOT NULL** a un parámetro.
- ✓ No podemos especificar una restricción del tipo:

```
PROCEDURE KK(a NUMBER(10)) IS ...     --ilegal
```

- ✓ Una función siempre debe acabar con la sentencia **RETURN**.

Podemos definir subprogramas al final de la parte declarativa de cualquier bloque. En Oracle, cualquier identificador debe estar declarado antes de usarse, y eso mismo pasa con los subprogramas, por lo que deberemos declararlos antes de usarlos.

```
DECLARE
hijos NUMBER;
FUNCTION hijos_familia( id_familia NUMBER )
    RETURN NUMBER IS
    hijos NUMBER;
BEGIN
    SELECT COUNT(*) INTO hijos FROM agentes
        WHERE familia = id_familia;
    RETURN hijos;
END hijos_familia;
BEGIN
...
END;
```

Si quisiéramos definir subprogramas en orden alfabético o lógico, o necesitamos definir subprogramas mutuamente recursivos (uno llama a otro, y éste a su vez llama al anterior), deberemos usar la definición hacia delante, para evitar errores de compilación.

```
DECLARE
    PROCEDURE calculo(...);          --declaración hacia delante
    --Definimos subprogramas agrupados lógicamente
    PROCEDURE inicio(...) IS
    BEGIN
        ...
        calculo(...);
        ...
    END;
    ...
BEGIN
    ...
END;
```

Autoevaluación

Una función siempre debe devolver un valor.

○ Verdadero.
○ Falso.

En PL/SQL no podemos definir subprogramas mutuamente recursivos.

○ Verdadero.
○ Falso.

4.1.1.- Almacenar subprogramas en la base de datos.

Para almacenar un subprograma en la base de datos utilizaremos la misma sintaxis que para declararlo, anteponiendo **CREATE [OR REPLACE]** a **PROCEDURE** o **FUNCTION**, y finalizando el subprograma con una línea que simplemente contendrá el carácter '/' para indicarle a Oracle que termina ahí. Si especificamos **OR REPLACE** y el subprograma ya existía, éste será reemplazado. Si no lo especificamos y el subprograma ya existe, Oracle nos devolverá un error indicando que el nombre ya está siendo utilizado por otro objeto de la base de datos.

```
CREATE OR REPLACE FUNCTION hijos_familia(id_familia NUMBER)
RETURN NUMBER IS
hijos NUMBER;
BEGIN
SELECT COUNT(*) INTO hijos FROM agentes
WHERE familia = id_familia;
RETURN hijos;
END;
/
```

Cuando los subprogramas son almacenados en la base de datos, para ellos no podemos utilizar las declaraciones hacia delante, por lo que cualquier subprograma almacenado en la base de datos deberá conocer todos los subprogramas que utilice.

Para invocar un subprograma usaremos la sintaxis:

```
nombre_procedimiento [(parametro [,parametro] ...)];
variable := nombre_funcion [(parametro [, parametro] ...)];

BEGIN
...
hijos := hijos_familia(10);
...
END;
```

Si el subprograma está almacenado en la base de datos y queremos invocarlo desde SQL*Plus usaremos la sintaxis:

```
EXECUTE nombre_procedimiento [(parametros)];
EXECUTE :variable_sql := nombre_funcion [(parametros)];
```

Cuando almacenamos un subprograma en la base de datos éste es compilado antes. Si hay algún error se nos informará de los mismos y deberemos corregirlos por medio de la cláusula **OR REPLACE**, antes de que el subprograma pueda ser utilizado.

Hay varias vistas del diccionario de datos que nos ayudan a llevar un control de los subprogramas, tanto para ver su código, como los errores de compilación. También hay algunos comandos de SQL*Plus que nos ayudan a hacer lo mismo pero de forma algo menos engorrosa.

Vistas y comandos asociados a los subprogramas.

Información almacenada.	Vista del diccionario.	Comando.
Código fuente.	`USER_SOURCE`	`DESCRIBE`
Errores de compilación.	`USER_ERRORS`	`SHOW ERRORS`
Ocupación de memoria.	`USER_OBJECT_SIZE`	

También existe la vista **USER_OBJECTS** de la cual podemos obtener los nombres de todos los subprogramas almacenados.

Debes conocer

En la siguiente presentación te mostramos las vistas relacionadas con los subprogramas, la compilación de un subprograma con errores y cómo mostrar los mismos, la compilación de un subprograma correctamente y cómo mostrar su código fuente y la ejecución de un subprograma desde SQL*Plus.

Resumen textual alternativo

Autoevaluación

Una vez que hemos almacenado un subprograma en la base de datos podemos consultar su código mediante la vista USER_OBJECTS.

○ Verdadero.
○ Falso.

4.1.2.- Parámetros de los subprogramas.

Ahora vamos a profundizar un poco más en los parámetros que aceptan los subprogramas y cómo se los podemos pasar a la hora de invocarlos.

> Las variables pasadas como parámetros a un subprograma son llamadas **parámetros actuales**. Las variables referenciadas en la especificación del subprograma como parámetros, son llamadas **parámetros formales**.

Cuando llamamos a un subprograma, los parámetros actuales podemos escribirlos utilizando notación posicional o notación nombrada, es decir, la asociación entre parámetros actuales y formales podemos hacerla por posición o por nombre.

En la notación posicional, el primer parámetro actual se asocia con el primer parámetro formal, el segundo con el segundo, y así para el resto. En la notación nombrada usamos el operador => para asociar el parámetro actual al parámetro formal. También podemos usar notación mixta.

Los parámetros pueden ser de **entrada** al subprograma, de **salida**, o de **entrada/salida**. Por defecto si a un parámetro no le especificamos el modo, éste será de entrada. Si el parámetro es de salida o de entrada/salida, el parámetro actual debe ser una variable.

Un parámetro de entrada permite que pasemos valores al subprograma y no puede ser modificado en el cuerpo del subprograma. El parámetro actual pasado a un subprograma como parámetro formal de entrada puede ser una constante o una variable.

Un parámetro de salida permite devolver valores y dentro del subprograma actúa como variable no inicializada. El parámetro formal debe ser siempre una variable.

Un parámetro de entrada-salida se utiliza para pasar valores al subprograma y/o para recibirlos, por lo que un parámetro formal que actúe como parámetro actual de entrada-salida siempre deberá ser una variable.

Los parámetros de entrada los podemos inicializar a un valor por defecto. Si un subprograma tiene un parámetro inicializado con un valor por defecto, podemos invocarlo prescindiendo del parámetro y aceptando el valor por defecto o pasando el parámetro y sobrescribiendo el valor por defecto. Si queremos prescindir de un parámetro colocado entre medias de otros, deberemos usar notación nombrada o si los parámetros restantes también tienen valor por defecto, omitirlos todos.

Debes conocer

En el siguiente enlace puedes encontrar ejemplos sobre el paso de parámetros a nuestros subprogramas.

Paso de parámetros a subprogramas.

Autoevaluación

Indica de entre las siguientes afirmaciones las que creas que son correctas.

☐ En PL/SQL podemos usar la notación posicional para pasar parámetros.
☐ No existen los parámetros de salida ya que para eso existen las funciones.
☐ Los parámetros de entrada los podemos inicializar a un valor por defecto.

Mostrar Información

4.1.3.- Sobrecarga de subprogramas y recursividad.

PL/SQL también nos ofrece la posibilidad de sobrecargar funciones o procedimientos, es decir, llamar con el mismo nombre subprogramas que realizan el mismo cometido y que aceptan distinto número y/o tipo de parámetros. No podemos sobrecargar subprogramas que aceptan el mismo número y tipo de parámetros y sólo difieren en el modo. Tampoco podemos sobrecargar subprogramas con el mismo número de parámetros y que los tipos de los parámetros sean diferentes, pero de la misma familia, o sean subtipos basados en la misma familia.

Debes conocer

En el siguiente enlace podrás ver un ejemplo de una función que es sobrecarga tres veces dependiendo del tipo de parámetros que acepta.

Sobrecarga de subprogramas.

PL/SQL también nos ofrece la posibilidad de utilizar la recursión en nuestros subprogramas. Un subprograma es recursivo si éste se invoca a él mismo.

Debes conocer

En el siguiente enlace podrás ampliar información sobre la recursividad.

Recursividad.

En el siguiente enlace podrás ver un ejemplo del uso de la recursividad en nuestros subprogramas.

Ejemplo del uso de la recursividad.

Autoevaluación

En PL/SQL no podemos sobrecargar subprogramas que aceptan el mismo número y tipo de parámetros, pero sólo difieren en el modo.

○ Verdadero.
○ Falso.

En PL/SQL no podemos utilizar la recursión y tenemos que imitarla mediante la iteración.

○ Verdadero.
○ Falso.

4.2.- Paquetes.

Un paquete es un objeto que agrupa tipos, elementos y subprogramas. Suelen tener dos partes: la especificación y el cuerpo, aunque algunas veces el cuerpo no es necesario.

En la parte de especificación declararemos la interfaz del paquete con nuestra aplicación y en el cuerpo es donde implementaremos esa interfaz.

Para crear un paquete usaremos la siguiente sintaxis:

```
CREATE [OR REPLACE] PACKAGE nombre AS
     [declaraciones públicas y especificación de subprogramas]
END [nombre];

CREATE [OR REPLACE] PACKAGE BODY nombre AS
     [declaraciones privadas y cuerpo de los subprogramas especificados]
[BEGIN
     sentencias de inicialización]
END [nombre];
```

La parte de inicialización sólo se ejecuta una vez, la primera vez que el paquete es referenciado.

Debes conocer

En el siguiente enlace te mostramos un ejemplo de un paquete que agrupa las principales tareas que realizamos con nuestra base de datos de ejemplo.

Ejemplo de paquete.

Para referenciar las partes visibles de un paquete, lo haremos por medio de la notación del punto.

```
BEGIN
     ...
     call_center.borra_agente( 10 );
     ...
END;
```

Para saber más

Oracle nos suministra varios paquetes para simplificarnos algunas tareas. En el siguiente enlace puedes encontrar más información sobre los mismos.

Paquetes suministrados por Oracle.

4.2.1.- Ejemplos de utilización del paquete DBMS_OUTPUT.

Oracle nos suministra un paquete público con el cual podemos enviar mensajes desde subprogramas almacenados, paquetes y disparadores, colocarlos en un buffer y leerlos desde otros subprogramas almacenados, paquetes o disparadores.

SQL*Plus permite visualizar los mensajes que hay en el buffer, por medio del comando **SET SERVEROUTPUT ON**. La utilización fundamental de este paquete es para la depuración de nuestros subprogramas.

Veamos uno a uno los subprogramas que nos suministra este paquete:

- ✓ Habilita las llamadas a los demás subprogramas. No es necesario cuando está activada la opción **SERVEROUTPUT**. Podemos pasarle un parámetro indicando el tamaño del buffer.

```
ENABLE
ENABLE( buffer_size IN INTEGER DEFAULT 2000);
```

- ✓ Deshabilita las llamadas a los demás subprogramas y purga el buffer. Como con **ENABLE** no es necesario si estamos usando la opción **SERVEROUTPUT**.

```
DISABLE
DISABLE();
```

- ✓ Coloca elementos en el buffer, los cuales son convertidos a **VARCHAR2**.

```
PUT
PUT(item IN NUMBER);
PUT(item IN VARCHAR2);
PUT(item IN DATE);
```

- ✓ Coloca elementos en el buffer y los termina con un salto de línea.

```
PUT_LINE
PUT_LINE(item IN NUMBER);
PUT_LINE(item IN VARCHAR2);
PUT_LINE(item IN DATE);
```

- ✓ Coloca un salto de línea en el buffer. Utilizado cuando componemos una línea usando varios **PUT**.

```
NEW_LINE
NEW_LINE();
```

- ✓ Lee una línea del buffer colocándola en el parámetro **line** y obviando el salto de línea. El parámetro **status** devolverá 0 si nos hemos traído alguna línea y 1 en caso contrario.

```
GET_LINE
GET_LINE(line OUT VARCHAR2, status OUT VARCHAR2);
```

- ✓ Intenta leer el número de líneas indicado en **numlines**. Una vez ejecutado, **numlines** contendrá el número de líneas que se ha traído. Las líneas traídas las coloca en el parámetro **lines** del tipo **CHARARR**, tipo definido el paquete **DBMS_OUTPUT** como una tabla de **VARCHAR2(255)**.

```
GET_LINES
GET_LINES(lines OUT CHARARR, numlines IN OUT INTEGER);
```

Ejercicio resuelto

Debes crear un procedimiento que visualice todos los agentes, su nombre, nombre de la familia y/o nombre de la oficina a la que pertenece.

4.3.- Objetos.

Hoy día, la programación orientada a objetos es uno de los paradigmas más utilizados y casi todos los lenguajes de programación la soportan. En este apartado vamos a dar unas pequeñas pinceladas de su uso en PL/SQL que serán ampliados en la siguiente unidad de trabajo.

Un tipo de objeto es un tipo de dato compuesto, que encapsula unos datos y las funciones y procedimientos necesarios para manipular esos datos. Las variables son los atributos y los subprogramas son llamados métodos. Podemos pensar en un tipo de objeto como en una entidad que posee unos atributos y un comportamiento (que viene dado por los métodos).

- Cuando creamos un tipo de objeto, lo que estamos creando es una entidad abstracta que especifica los atributos que tendrán los objetos de ese tipo y define su comportamiento.
- Cuando instanciamos un objeto estamos particularizando la entidad abstracta a una en particular, con los atributos que tiene el tipo de objeto, pero con un valor dado y con el mismo comportamiento.

Los tipos de objetos tiene 2 partes: una **especificación** y un **cuerpo**. La parte de especificación declara los atributos y los métodos que harán de interfaz de nuestro tipo de objeto. En el cuerpo se implementa la parte de especificación. En la parte de especificación debemos declarar primero los atributos y después los métodos. Todos los atributos son públicos (visibles). No podemos declarar atributos en el cuerpo, pero sí podemos declarar subprogramas locales que serán visibles en el cuerpo del objeto y que nos ayudarán a implementar nuestros métodos.

Los atributos pueden ser de cualquier tipo de datos Oracle, excepto:

- **LONG** y **LONG RAW**.
- **NCHAR**, **NCLOB** y **NVARCHAR2**.
- **MLSLABEL** y **ROWID**.
- Tipos específicos de PL/SQL: **BINARY_INTEGER**, **BOOLEAN**, **PLS_INTEGER**, **RECORD**, **REF CURSOR**, **%TYPE** y **%ROWTYPE**.
- Tipos definidos dentro de un paquete PL/SQL.

No podemos inicializar un atributo en la declaración. Tampoco podemos imponerle la restricción **NOT NULL**.

Un **método** es un subprograma declarado en la parte de especificación de un tipo de objeto por medio de: **MEMBER**. Un método no puede llamarse igual que el tipo de objeto o que cualquier atributo. Para cada método en la parte de especificación, debe haber un método implementado en el cuerpo con la misma cabecera.

Todos los métodos en un tipo de objeto aceptan como primer parámetro una instancia de su tipo. Este parámetro es **SELF** y siempre está accesible a un método. Si lo declaramos explícitamente debe ser el primer parámetro, con el nombre **SELF** y del tipo del tipo de objeto. Si **SELF** no está declarado explícitamente, por defecto será **IN** para las funciones e **IN OUT** para los procedimientos.

Los métodos dentro de un tipo de objeto pueden sobrecargarse. No podemos sobrecargarlos si los parámetros formales sólo difieren en el modo o pertenecen a la misma familia. Tampoco podremos sobrecargar una función miembro si sólo difiere en el tipo devuelto.

Una vez que tenemos creado el objeto, podemos usarlo en cualquier declaración. Un objeto cuando se declara sigue las mismas reglas de alcance y visibilidad que cualquier otra variable.

Cuando un objeto se declara éste es automáticamente **NULL**. Dejará de ser nulo cuando lo inicialicemos por medio de su constructor o cuando le asignemos otro. Si intentamos acceder a los atributos de un objeto **NULL** saltará la excepción **ACCES_INTO_NULL**.

Todos los objetos tienen constructores por defecto con el mismo nombre que el tipo de objeto y acepta tantos parámetros como atributos del tipo de objeto y con el mismo tipo. PL/SQL no llama implícitamente a los constructores, deberemos hacerlo nosotros explícitamente.

```
DECLARE
    familia1 Familia;
BEGIN
    ...
    familia1 := Familia( 10, 'Fam10', 1, NULL );
    ...
END;
```

Un tipo de objeto puede tener a otro tipo de objeto entre sus atributos. El tipo de objeto que hace de atributo debe ser conocido por Oracle. Si 2 tipos de objetos son mutuamente dependientes, podemos usar una declaración hacia delante para evitar errores de compilación.

Ejercicio resuelto

Cómo declararías los objetos para nuestra base de datos de ejemplo.

4.3.1.- Objetos. Funciones mapa y funciones de orden.

En la mayoría de los problemas es necesario hacer comparaciones entre tipos de datos, ordenarlos, etc. Sin embargo, los tipos de objetos no tienen orden predefinido por lo que no podrán ser comparados ni ordenados (<>, **DISTINCT**, **ORDER BY**, ...). Nosotros podemos definir el orden que seguirá un tipo de objeto por medio de las funciones mapa y las funciones de orden.

Una función miembro mapa es una función sin parámetros que devuelve un tipo de dato: **DATE**, **NUMBER** o **VARCHAR2** y sería similar a una función hash. Se definen anteponiendo la palabra clave **MAP** y sólo puede haber una para el tipo de objeto.

```
CREATE TYPE Familia AS OBJECT (
identificador      NUMBER,
nombre           VARCHAR2(20),
familia_          NUMBER,
oficina_          NUMBER,
MAP MEMBER FUNCTION orden RETURN NUMBER,
...
);

CREATE TYPE BODY Familia AS
MAP MEMBER FUNCTION orden RETURN NUMBER IS
BEGIN
    RETURN identificador;
END;
...
END;
```

Una función miembro de orden es una función que acepta un parámetro del mismo tipo del tipo de objeto y que devuelve un número negativo si el objeto pasado es menor, cero si son iguales y un número positivo si el objeto pasado es mayor.

```
CREATE TYPE Oficina AS OBJECT (
    identificador          NUMBER,
    nombre               VARCHAR2(20),
    ...
    ORDER MEMBER FUNCTION igual ( ofi Oficina ) RETURN INTEGER,
    ...
);

CREATE TYPE BODY Oficina AS
    ORDER MEMBER FUNCTION igual ( ofi Oficina ) RETURN INTEGER IS
    BEGIN
        IF (identificador < ofi.identificador) THEN
            RETURN -1;
        ELSIF (identificador = ofi.identificador) THEN
            RETURN 0;
        ELSE
            RETURN 1;
        END IF;
    END;
    ...
END;
```

Autoevaluación

Los métodos de un objeto sólo pueden ser procedimientos.

○ Verdadero.
○ Falso.

El orden de un objeto se consigue:

○ Al crearlo.
○ En PL/SQL los objetos no pueden ser ordenados.
○ Mediante las funciones mapa y las funciones de orden.

5.- Disparadores.

Caso práctico

Juan y **María** ya han hecho un paquete en el que tienen agrupadas las funciones más usuales que realizan sobre la base de datos de juegos on-line. Sin embargo, **María** ve que hay cosas que aún no pueden controlar. Por ejemplo, **María** quiere que la clave y el usuario de un jugador o jugadora no puedan ser la misma y eso no sabe cómo hacerlo. **Juan** le dice que para ese cometido están los disparadores y sin más dilaciones se pone a explicarle a **María** para qué sirven y cómo utilizarlos.

En este apartado vamos a tratar una herramienta muy potente proporcionada por PL/SQL para programar nuestra base de datos que son los disparadores o triggers en inglés.

Pero, ¿qué es un disparador?

Un disparador no es más que un procedimiento que es ejecutado cuando se realiza alguna sentencia de manipulación de datos sobre una tabla dada y bajo unas circunstancias establecidas a la hora de definirlo.

Por lo que un disparador puede ser usado para:

- ✓ Llevar a cabo auditorías sobre la historia de los datos en nuestra base de datos.
- ✓ Garantizar complejas reglas de integridad.
- ✓ Automatizar la generación de valores derivados de columnas.
- ✓ Etc.

Cuando diseñamos un disparador debemos tener en cuenta que:

- ✓ No debemos definir disparadores que dupliquen la funcionalidad que ya incorpora Oracle.
- ✓ Debemos limitar el tamaño de nuestros disparadores, y si estos son muy grandes codificarlos por medio de subprogramas que sean llamados desde el disparador.
- ✓ Cuidar la creación de disparadores recursivos.

Un disparador puede ser lanzado antes o después de realizar la operación que lo lanza. Por lo que tendremos disparadores **BEFORE** y disparadores **AFTER**.

Un disparador puede ser lanzado una vez por sentencia o una vez por cada fila a la que afecta. Por lo que tendremos disparadores de sentencia y disparadores de fila.

Un disparador puede ser lanzado al insertar, al actualizar o al borrar de una tabla, por lo que tendremos disparadores **INSERT**, **UPDATE** o **DELETE** (o mezclados).

Autoevaluación

En PL/SQL sólo podemos definir disparadores de fila.

Verdadero.

○ Falso.

La diferente entre un disparador de fila y uno de sentencia es que el de fila es lanzado una vez por fila a la que afecta la sentencia y el de sentencia es lanzado una sola vez.

○ Verdadero.

○ Falso.

5.1.- Definición de disparadores.

Por lo visto anteriormente, para definir un disparador deberemos indicar si será lanzado antes o después de la ejecución de la sentencia que lo lanza, si se lanzará una vez por sentencia o una vez por fila a la que afecta, y si será lanzado al insertar y/o al actualizar y/o al borrar. La sintaxis que seguiremos para definir un disparador será la siguiente:

```
CREATE [OR REPLACE] TRIGGER nombre
momento acontecimiento ON tabla
[[REFERENCING (old AS alias_old|new AS alias_new)
FOR EACH ROW
[WHEN condicion]]
bloque_PL/SQL;
```

Donde nombre nos indica el **nombre** que le damos al disparador, **momento** nos dice cuando será lanzado el disparador (**BEFORE** o **AFTER**), **acontecimiento** será la acción que provoca el lanzamiento del disparador (**INSERT** y/o **DELETE** y/o **UPDATE**). **REFERENCING** y **WHEN** sólo podrán ser utilizados con disparadores para filas. **REFERENCING** nos permite asignar un alias a los valores **NEW** o/y **OLD** de las filas afectadas por la operación, y **WHEN** nos permite indicar al disparador que sólo sea lanzado cuando sea **TRUE** una cierta condición evaluada para cada fila afectada.

En un disparador de fila, podemos acceder a los valores antiguos y nuevos de la fila afectada por la operación, referenciados como **old** y **new** (de ahí que podamos utilizar la opción **REFERENCING** para asignar un alias). Si el disparador es lanzado al insertar, el valor antiguo no tendrá sentido y el valor nuevo será la fila que estamos insertando. Para un disparador lanzado al actualizar el valor antiguo contendrá la fila antes de actualizar y el valor nuevo contendrá la fila que vamos actualizar. Para un disparador lanzado al borrar sólo tendrá sentido el valor antiguo.

En el cuerpo de un disparador también podemos acceder a unos predicados que nos dicen qué tipo de operación se está llevando a cabo, que son: **INSERTING**, **UPDATING** y **DELETING**.

Un disparador de fila no puede acceder a la tabla asociada. Se dice que esa tabla está mutando. Si un disparador es lanzado en cascada por otro disparador, éste no podrá acceder a ninguna de las tablas asociadas, y así recursivamente.

```
CREATE TRIGGER prueba BEFORE UPDATE ON agentes
FOR EACH ROW
BEGIN
     ...
     SELECT identificador FROM agentes WHERE ...
/*devolvería el error ORA-04091: table AGENTES is mutating, trigger/function may not see it*/
     ...
END;
/
```

Si tenemos varios tipos de disparadores sobre una misma tabla, el orden de ejecución será:

- ✓ Triggers before de sentencia.
- ✓ Triggers before de fila.
- ✓ Triggers after de fila.
- ✓ Triggers after de sentencia.

Existe una vista del diccionario de datos con información sobre los disparadores: **USER_TRIGGERS**;

```
SQL>DESC USER_TRIGGERS;
Name                           Null?    Type
------------------------------ -------- ----
TRIGGER_NAME                   NOT NULL VARCHAR2(30)
TRIGGER_TYPE                            VARCHAR2(16)
TRIGGERING_EVENT                        VARCHAR2(26)
TABLE_OWNER                    NOT NULL VARCHAR2(30)
TABLE_NAME                     NOT NULL VARCHAR2(30)
REFERENCING_NAMES                       VARCHAR2(87)
WHEN_CLAUSE                             VARCHAR2(4000)
STATUS                                  VARCHAR2(8)
DESCRIPTION                             VARCHAR2(4000)
TRIGGER_BODY                            LONG
```

5.2.- Ejemplos de disparadores.

Ya hemos visto qué son los disparadores, los tipos que existen, cómo definirlos y algunas consideraciones a tener en cuenta a la hora de trabajar con ellos.

Ahora vamos a ver algunos ejemplos de su utilización con los que podremos comprobar la potencia que éstos nos ofrecen.

Ejercicio resuelto

Como un agente debe pertenecer a una familia o una oficina pero no puede pertenecer a una familia y a una oficina a la vez, deberemos implementar un disparador para llevar a cabo esta restricción que Oracle no nos permite definir.

Debes conocer

En la siguiente presentación podrás ver la descripción de la tabla USER_TRIGGERS, cómo se almacena un disparador en la base de datos, cómo se consulta su código y cómo se lanza su ejecución.

Resumen textual alternativo

Ejercicio resuelto

Supongamos que tenemos una tabla de históricos para agentes que nos permita auditar las familias y oficinas por la que ha ido pasando un agente. La tabla tiene la fecha de inicio y la fecha de finalización del agente en esa familia u oficina, el identificador del agente, el nombre del agente, el nombre de la familia y el nombre de la oficina. Queremos hacer un disparador que inserte en esa tabla.

Ejercicio resuelto

Queremos realizar un disparador que no nos permita llevar a cabo operaciones con familias si no estamos en la jornada laboral.

6.- Interfaces de programación de aplicaciones para lenguajes externos.

Caso práctico

Juan y **María** tienen la base de datos de juegos on-line lista, pero no tienen claro si todo lo que han hecho lo tienen que utilizar desde dentro de la base de datos o si pueden reutilizar todo ese trabajo desde otro lenguaje de programación desde el que están creando la interfaz web desde la que los jugadores y jugadoras se conectarán para jugar y desde la que también se conectará la administradora para gestionar la base de datos de una forma más amigable.

En ese momento pasa **Ada** al lado y ambos la asaltan con su gran duda. **Ada**, muy segura, les responde que para eso existen las interfaces de programación de aplicaciones o APIs, que les permiten acceder desde otros lenguajes de programación a la base de datos.

Llegados a este punto ya sabemos programar nuestra base de datos, pero al igual que a **Juan** y **María** te surgirá la duda de si todo lo que has hecho puedes aprovecharlo desde cualquier otro lenguaje de programación. La respuesta es la misma que dio **Ada**: SÍ.

En este apartado te vamos a dar algunas referencias sobre las APIs para acceder a las bases de datos desde un lenguaje de programación externo. No vamos a ser exhaustivos ya que eso queda fuera del alcance de esta unidad e incluso de este módulo. Todo ello lo vas a estudiar con mucha más profundidad en otros módulos de este ciclo (o incluso ya lo conoces). Aquí sólo pretendemos que sepas que existen y que las puedes utilizar.

Las primeras APIs utilizadas para acceder a bases de datos Oracle fueron las que el mismo Oracle proporcionaba: Pro*C, Pro*Fortran y Pro*Cobol. Todas permitían embeber llamadas a la base de datos en nuestro programa y a la hora de compilarlo, primero debíamos pasar el precompilador adecuado que trasladaba esas llamadas embebidas en llamadas a una librería utilizada en tiempo de ejecución. Sin duda, el más utilizado fue Pro*C, ya que el lenguaje C y C++ tuvieron un gran auge.

Para saber más

En los siguientes enlaces podrás obtener más información sobre Pro*C y cómo crear un programa para acceder a nuestra base de datos.

Introducción a Pro*C.

Ejemplo de programa en Pro*C.

Hoy día existen muchas más APIs para acceder a las bases de datos ya que tanto los lenguajes de programación como las tecnologías han evolucionado mucho. Antes la programación para la web casi no existía y por eso todos los programas que accedían a bases de datos lo hacían bien en local o bien a través de una red local. Hoy día eso sería impensable, de ahí que las APIs también hayan evolucionado mucho y tengamos una gran oferta a nuestro alcance para elegir la que más se adecue a nuestras necesidades.

Para saber más

En los siguientes enlaces podrás ampliar información sobre algunas APIs muy comunes para acceder a bases de datos.

Tecnologías Java para acceder a bases de datos.

Conectividad abierta de bases de datos.

Ejemplo de conexión desde Java a Oracle y a MySQL.

Anexo I.- Caso de estudio.

Una empresa de telefonía tiene sus centros de llamadas distribuidos por la geografía española en diferentes oficinas. Estas oficinas están jerarquizadas en familias de agentes telefónicos. Cada familia, por tanto, podrá contener agentes u otras familias. Los agentes telefónicos, según su categoría, además se encargarán de supervisar el trabajo de todos los agentes de una oficina o de coordinar el trabajo de los agentes de una familia dada. El único agente que pertenecerá directamente a una oficina y que no formará parte de ninguna familia será el supervisor de dicha oficina, cuya categoría es la 2. Los coordinadores de las familias deben pertenecer a dicha familia y su categoría será 1 (no todas las familias tienen por qué tener un coordinador y dependerá del tamaño de la oficina, ya que de ese trabajo también se puede encargar el supervisor de la oficina). Los demás agentes deberán pertenecer a una familia, su categoría será 0 y serán los que principalmente se ocupen de atender las llamadas.

- ✓ De los agentes queremos conocer su nombre, su clave y contraseña para entrar al sistema, su categoría y su habilidad que será un número entre 0 y 9 indicando su habilidad para atender llamadas.
- ✓ Para las familias sólo nos interesa conocer su nombre.
- ✓ Finalmente, para las oficinas queremos saber su nombre, domicilio, localidad y código postal de la misma.

Un posible modelo entidad-relación para el problema expuesto podría ser el siguiente:

De este modelo de datos surgen tres tablas, que podrían ser creadas en Oracle con las siguientes sentencias:

```
create table oficinas (
    identificador     number(6) not null primary key,
    nombre          varchar2(40) not null unique,
    domicilio     varchar2(40),
    localidad     varchar2(20),
    codigo_postal     number(5)
);

create table familias (
    identificador     number(6) not null primary key,
    nombre          varchar2(40) not null unique,
    familia          number(6) references familias,
    oficina          number(6) references oficinas
);

create table agentes (
    identificador     number(6) not null primary key,
    nombre          varchar2(60) not null,
    usuario          varchar2(20) not null unique,
    clave          varchar2(20) not null,
    habilidad     number(1) not null,
    categoria     number(1) not null,
    familia          number(6) references familias,
    oficina          number(6) references oficinas
);
```

Vamos a insertar algunas filas para que las pruebas de nuestros ejemplos tengan algo de sentido. Para ello podemos utilizar las siguientes sentencias:

```
insert into oficinas values (1, 'Madrid', 'Gran vía, 37', 'Madrid', 28000);
insert into oficinas values (2, 'Granada', 'Camino Ronda, 50', 'Granada', 36000);
insert into oficinas values (3, 'Jaén', 'Gran Eje, 80', 'Jaén', 27000);

insert into familias values (11, 'Madrid-1', NULL, 1);
insert into familias values (111, 'Madrid-1.1', 11, NULL);
insert into familias values (112, 'Madrid-1.2', 11, NULL);
insert into familias values (1121, 'Madrid-1.2.1', 112, NULL);
insert into familias values (1122, 'Madrid-1.2.2', 112, NULL);
insert into familias values (1123, 'Madrid-1.2.3', 112, NULL);
insert into familias values (21, 'Granada-1', NULL, 2);
insert into familias values (211, 'Granada-1.1', 21, NULL);
insert into familias values (212, 'Granada-1.2', 21, NULL);
insert into familias values (213, 'Granada-1.3', 21, NULL);
insert into familias values (31, 'Jaén-1', NULL, 3);

insert into agentes values (31, 'José Ramón Jiménez Reyes', 'jrjr', 'sup31', 9, 2, NULL, 3);
insert into agentes values (311, 'Pedro Fernández Arias', 'pfa', 'ag311', 5, 0, 31, NULL);
insert into agentes values (312, 'Vanesa Sánchez Rojo', 'vsr', 'ag312', 5, 0, 31, NULL);
insert into agentes values (313, 'Francisco Javier García Escobedo', 'fjge', 'ag313', 5, 0, 31, NULL);
insert into agentes values (314, 'Pilar Ramirez Pérez', 'prp', 'ag314', 5, 0, 31, NULL);
insert into agentes values (315, 'José Luis García Martínez', 'jlgm', 'ag315', 5, 0, 31, NULL);
insert into agentes values (21, 'Sebastián López Ojeda', 'slo', 'sup21', 9, 2, NULL, 2);
insert into agentes values (211, 'Diosdado Sánchez Hernández', 'dsh', 'ag211', 8, 1, 21, NULL);
insert into agentes values (2111, 'José Juan Cano Pardo', 'jjcp', 'ag2111', 5, 0, 211, NULL);
insert into agentes values (2112, 'Flor Moncada Añón', 'ag2112', 'fma', 5, 0, 211, NULL);
insert into agentes values (2113, 'Juan Manuel Alcazar Donaire', 'jmad', 'ag2113', 5, 0, 211, NULL);
insert into agentes values (2121, 'Manuel Jesús Rubia Mateos', 'mjrm', 'ag2121', 5, 0, 212, NULL);
insert into agentes values (2122, 'Esther López Delgado', 'eld', 'ag2122', 5, 0, 212, NULL);
insert into agentes values (2123, 'Francisco Javier Cabrerizo Membrilla', 'fjcm', 'ag2123', 5, 0, 212, NULL);
insert into agentes values (2124, 'Verónica Cabrerizo Menbrilla', 'vcm', 'ag2124', 5, 0, 212, NULL);
insert into agentes values (2125, 'María José Navascués Morales', 'mjnm', 'ag2125', 5, 0, 212, NULL);
insert into agentes values (2131, 'Isabel Cruz Granados', 'icg', 'ag2131', 5, 0, 213, NULL);
insert into agentes values (2132, 'Antonio Casado Fernández', 'acf', 'ag2132', 5, 0, 213, NULL);
insert into agentes values (2133, 'Gabriel Callejón García', 'gcg', 'ag2133', 5, 0, 213, NULL);
insert into agentes values (2134, 'Enrique Cano Balsera', 'ecb', 'ag2134', 5, 0, 213, NULL);
insert into agentes values (11, 'Narciso Jáimez Toro', 'njt', 'sup11', 9, 2, NULL, 1);
insert into agentes values (111, 'Jesús Baños Sancho', 'jbs', 'ag111', 8, 1, 11, NULL);
insert into agentes values (1111, 'Salvador Romero Villegas', 'srv', 'ag1111', 7, 1, 111, NULL);
insert into agentes values (1112, 'José Javier Bermúdez Hernández', 'jjbh', 'ag1112', 7, 1, 111, NULL);
insert into agentes values (1113, 'Alfonso Bonillo Sierra', 'abs', 'ag1113', 7, 1, 111, NULL);
insert into agentes values (1121, 'Silvia Thomas Barrós', 'stb', 'ag1121', 7, 1, 112, NULL);
insert into agentes values (11211, 'Ernesto Osoro Gorrotxategi', 'eog', 'ag11211', 5, 0, 1121, NULL);
insert into agentes values (11212, 'Guillermo Campos Guillén', 'gcg', 'ag11212', 5, 0, 1121, NULL);
insert into agentes values (11213, 'Antonio Fernández Ruíz', 'afr', 'ag11213', 5, 0, 1121, NULL);
insert into agentes values (11214, 'María Luisa López Caballero', 'mllc', 'ag11214', 5, 0, 1121, NULL);
insert into agentes values (11221, 'Virginia Morenas Rubio', 'vmr', 'ag11221', 5, 0, 1121, NULL);
insert into agentes values (11222, 'Rosario Castro García', 'rcg', 'ag11222', 5, 0, 1122, NULL);
insert into agentes values (11223, 'Antonio Álvarez Palomeque', 'aap', 'ag11223', 5, 0, 1122, NULL);
insert into agentes values (11224, 'David Martínez Martínez', 'dmm', 'ag11224', 5, 0, 1122, NULL);
insert into agentes values (11225, 'Juan Corral González', 'jcg', 'ag11225', 5, 0, 1122, NULL);
insert into agentes values (11226, 'Eduardo Alfada Pereira', 'eap', 'ag11226', 5, 0, 1122, NULL);
insert into agentes values (11231, 'Cayetano García Herrera', 'cgh', 'ag11231', 5, 0, 1123, NULL);
insert into agentes values (11232, 'José Antonio Sieres Vega', 'jasv', 'ag11232', 5, 0, 1123, NULL);
insert into agentes values (11233, 'Juan Manuel Guzmán García', 'jmgg', 'ag11233', 5, 0, 1123, NULL);

commit;
```

Anexo II.- Excepciones predefinidas en Oracle.

Las excepciones predefinidas son:

Excepciones predefinidas en Oracle.

Excepción.	`SQLCODE`	Lanzada cuando ...
`ACCES_INTO_NULL`	-6530	Intentamos asignar valor a atributos de objetos no inicializados.
`COLECTION_IS_NULL`	-6531	Intentamos asignar valor a elementos de colecciones no inicializadas, o acceder a métodos distintos de `EXISTS`.
`CURSOR_ALREADY_OPEN`	-6511	Intentamos abrir un cursor ya abierto.
`DUP_VAL_ON_INDEX`	-1	Índice único violado.
`INVALID_CURSOR`	-1001	Intentamos hacer una operación con un cursor que no está abierto.
`INVALID_NUMBER`	-1722	Conversión de cadena a número falla.
`LOGIN_DENIED`	-1403	El usuario y/o contraseña para conectarnos a Oracle no es válido.
`NO_DATA_FOUND`	+100	Una sentencia `SELECT` no devuelve valores, o intentamos acceder a un elemento borrado de una tabla anidada.
`NOT_LOGGED_ON`	-1012	No estamos conectados a Oracle.
`PROGRAM_ERROR`	-6501	Ha ocurrido un error interno en PL/SQL.
`ROWTYPE_MISMATCH`	-6504	Diferentes tipos en la asignación de 2 cursores.
`STORAGE_ERROR`	-6500	Memoria corrupta.
`SUBSCRIPT_BEYOND_COUNT`	-6533	El índice al que intentamos acceder en una colección sobrepasa su límite superior.
`SUBSCRIPT_OUTSIDE_LIMIT`	-6532	Intentamos acceder a un rango no válido dentro de una colección (-1 por ejemplo).
`TIMEOUT_ON_RESOURCE`	-51	Un timeout ocurre mientras Oracle espera por un recurso.
`TOO_MANY_ROWS`	-1422	Una sentencia `SELECT ... INTO ...` devuelve más de una fila.
`VALUE_ERROR`	-6502	Ocurre un error de conversión, aritmético, de truncado o de restricción de tamaño.
`ZERO_DIVIDE`	-1476	Intentamos dividir un número por 0.

Anexo III - Evaluación de los atributos de un cursor explícito.

Evaluación de los atributos de un cursor explícito según las operaciones realizadas con él.

Operación realizada.	%FOUND	%NOTFOUND	%ISOPEN	%ROWCOUNT
Antes del OPEN	Excepción.	Excepción.	FALSE	Excepción.
Después del OPEN	NULL	NULL	TRUE	0
Antes del primer FETCH	NULL	NULL	TRUE	0
Después del primer FETCH	TRUE	FALSE	TRUE	1
Antes de los siguientes FETCH	TRUE	FALSE	TRUE	1
Después de los siguientes FETCH	TRUE	FALSE	TRUE	Depende datos.
Antes del último FETCH	TRUE	FALSE	TRUE	Depende datos.
Después del último FETCH	FALSE	TRUE	TRUE	Depende datos.
Antes del CLOSE	FALSE	TRUE	TRUE	Depende datos.
Después del CLOSE	Excepción.	Excepción.	FALSE	Excepción.

Anexo IV.- Paso de parámetros a subprogramas.

Veamos algunos ejemplos del paso de parámetros a subprogramas:

- ✓ Notación mixta.

```
DECLARE
    PROCEDURE prueba( formal1 NUMBER, formal2 VARCHAR2) IS
    BEGIN
        ...
    END;
    actual1 NUMBER;
    actual2 VARCHAR2;
BEGIN
    ...
    prueba(actual1, actual2);               --posicional
    prueba(formal2=>actual2,formal1=>actual1);     --nombrada
    prueba(actual1, formal2=>actual2);          --mixta
END;
```

- ✓ Parámetros de entrada.

```
    FUNCTION categoria( id_agente IN NUMBER )
    RETURN NUMBER IS
        cat NUMBER;
    BEGIN
        ...
        SELECT categoria INTO cat FROM agentes
    WHERE identificador = id_agente;
    RETURN cat;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            id_agente := -1; --ilegal, parámetro de entrada
    END;
```

- ✓ Parámetros de salida.

```
PROCEDURE nombre( id_agente NUMBER, nombre OUT VARCHAR2) IS
BEGIN
    IF (nombre = 'LUIS') THEN     --error de sintaxis
    END IF;
    ...
END;
```

- ✓ Parámetros con valor por defecto de los que podemos prescindir.

```
DECLARE
    SUBTYPE familia IS familias%ROWTYPE;
    SUBTYPE agente IS agentes%ROWTYPE;
    SUBTYPE tabla_agentes IS TABLE OF agente;
    familia1 familia;
    familia2 familia;
    hijos_fam tabla_agentes;
    FUNCTION inserta_familia( mi_familia familia,
        mis_agentes tabla_agentes := tabla_agentes() )
RETURN NUMBER IS
    BEGIN
        INSERT INTO familias VALUES (mi_familia);
        FOR i IN 1..mis_agentes.COUNT LOOP
            IF (mis_agentes(i).oficina IS NOT NULL) or (mis_agentes(i).familia != mi_familia.identificador) THEN
                ROLLBACK;
                RETURN -1;
            END IF;
            INSERT INTO agentes VALUES (mis_agentes(i));
        END LOOP;
        COMMIT;
        RETURN 0;
    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
            ROLLABACK;
            RETURN -1;
        WHEN OTHERS THEN
            ROLLBACK;
            RETURN SQLCODE;
    END inserta_familia;
BEGIN
    ...
    resultado := inserta_familia( familia1 );
    ...
    resultado := inserta_familia( familia2, hijos_fam2 );
    ...
END;
```

Anexo V.- Sobrecarga de subprogramas.

Veamos un ejemplo de sobrecarga de subprogramas en el que una misma función es sobrecargada tres veces para diferentes tipos de parámetros.

```
DECLARE
    TYPE agente IS agentes%ROWTYPE;
    TYPE familia IS familias%ROWTYPE;
    TYPE tAgentes IS TABLE OF agente;
    TYPE tFamilias IS TABLE OF familia;

    FUNCTION inserta_familia( mi_familia familia )
    RETURN NUMBER IS
    BEGIN
        INSERT INTO familias VALUES (mi_familia.identificador, mi_familia.nombre, mi_familia.familia, mi_familia.oficina );
        COMMIT;
        RETURN 0;
    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
            ROLLBACK;
            RETURN -1;
        WHEN OTHERS THEN
            ROLLBACK;
            RETURN SQLCODE;
    END inserta_familia;

    FUNCTION inserta_familia( mi_familia familia, hijas tFamilias )
    RETURN NUMBER IS
    BEGIN
        INSERT INTO familias VALUES (mi_famila.identificador, mi_familia.nombre, mi_familia.familia, mi_familia.oficina );
        IF (hijas IS NOT NULL) THEN
            FOR i IN 1..hijas.COUNT LOOP
                IF (hijas(i).oficina IS NOT NULL) or (hijas(i).familia != mi_familia.identificador) THEN
                    ROLLBACK;
                    RETURN -1;
                END IF;
                INSERT INTO familias VALUES (hijas(i).identificador, hijas(i).nombre, hijas(i).familia, hijas(i).oficina );
            END LOOP;
        END IF;
        COMMIT;
        RETURN 0;
    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
            ROLLBACK;
            RETURN -1;
        WHEN OTHERS THEN
            ROLLBACK;
            RETURN -1;
    END inserta_familia;

    FUNCTION inserta_familia( mi_familia familia, hijos tAgentes )
    RETURN NUMBER IS
    BEGIN
        INSERT INTO familias VALUES (mi_familia.identificador, mi_familia.nombre, mi_familia.familia, mi_familia.oficina );
        IF (hijos IS NOT NULL) THEN
            FOR i IN 1..hijos.COUNT LOOP
                IF (hijos(i).oficina IS NOT NULL) or (hijos(i).familia != mi_familia.identificador) THEN
                    ROLLBACK;
                    RETURN -1;
                END IF;
                INSERT INTO agentes VALUES (hijos(i).identificador, hijos(i).nombre, hijos(i).usuario, hijos(i).clave, hijos(i).habilidad, hijos(i).categoria, hijos(i).familia, hijos(i).oficina );
            END LOOP;
        END IF;
        COMMIT;
        RETURN 0;
    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
            ROLLBACK;
            RETURN -1;
        WHEN OTHERS THEN
            ROLLBACK;
            RETURN -1;
    END inserta_familias;

    mi_familia familia;
    mi_familia1 familia;
    familias_hijas tFamilias;
    mi_familia2 familia;
    hijos tAgentes;
BEGIN
    ...
    resultado := inserta_familia(mi_familia);
    ...
    resultado := inserta_familia(mi_familia1, familias_hijas);
    ...
    resultado := inserta_familia(mi_familia2, hijos);
    ...
END;
```

Anexo VI.- Ejemplo de recursividad.

Aquí tienes un ejemplo del uso de la recursividad en nuestros subprogramas.

```
DECLARE
    TYPE agente IS agentes%ROWTYPE;
    TYPE tAgentes IS TABLE OF agente;
    hijos10 tAgentes;
    PROCEDURE dame_hijos( id_familia NUMBER,
                hijos IN OUT tAgentes ) IS
        CURSOR hijas IS SELECT identificador FROM familias WHERE familia = id_familia;
hija NUMBER;
    CURSOR cHijos IS SELECT * FROM agentes WHERE familia = id_familia;
hijo agente;
    BEGIN
        --Si la tabla no está inicializada -> la inicializamos
        IF hijos IS NULL THEN
            hijos = tAgentes();
        END IF;
        --Metemos en la tabla los hijos directos de esta familia
        OPEN cHijos;
        LOOP
            FETCH cHijos INTO hijo;
            EXIT WHEN cHijos%NOTFOUND;
            hijos.EXTEND;
            hijos(hijos.LAST) := hijo;
        END LOOP;
        CLOSE cHijos;
        --Hacemos lo mismo para todas las familias hijas de la actual
        OPEN hijas;
        LOOP
            FETCH hijas INTO hija;
            EXIT WHEN hijas%NOTFOUND;
            dame_hijos( hija, hijos );
        END LOOP;
        CLOSE hijas;
    END dame_hijos;
BEGIN
    ...
    dame_hijos( 10, hijos10 );
    ...
END;
```

Anexo VII.- Ejemplo de paquete.

Aquí puedes ver un ejemplo de un paquete que agrupa las principales tareas que llevamos a cabo con la base de datos de ejemplo.

```
CREATE OR REPLACE PACKAGE call_center AS     --inicialización
     --Definimos los tipos que utilizaremos
     SUBTYPE agente IS agentes%ROWTYPE;
     SUBTYPE familia IS familias%ROWTYPE;
     SUBTYPE oficina IS oficinas%ROWTYPE;
     TYPE tAgentes IS TABLE OF agente;
     TYPE tFamilias IS TABLE OF familia;
     TYPE tOficinas IS TABLE OF oficina;

     --Definimos las excepciones propias
     referencia_no_encontrada exception;
     referencia_encontrada exception;
     no_null exception;
     PRAGMA EXCEPTION_INIT(referencia_no_encontrada, -2291);
     PRAGMA EXCEPTION_INIT(referencia_encontrada, -2292);
     PRAGMA EXCEPTION_INIT(no_null, -1400);

     --Definimos los errores que vamos a tratar
     todo_bien               CONSTANT NUMBER := 0;
     elemento_existente           CONSTANT NUMBER:= -1;
     elemento_inexistente           CONSTANT NUMBER:= -2;
     padre_existente          CONSTANT NUMBER:= -3;
     padre_inexistente          CONSTANT NUMBER:= -4;
     no_null_violado          CONSTANT NUMBER:= -5;
     operacion_no_permitida          CONSTANT NUMBER:= -6;

     --Definimos los subprogramas públicos
     --Nos devuelve la oficina padre de un agente
     PROCEDURE oficina_padre( mi_agente agente, padre OUT oficina );

     --Nos devuelve la oficina padre de una familia
     PROCEDURE oficina_padre( mi_familia familia, padre OUT oficina );

     --Nos da los hijos de una familia
     PROCEDURE dame_hijos( mi_familia familia, hijos IN OUT tAgentes );

     --Nos da los hijos de una oficina
     PROCEDURE dame_hijos( mi_oficina oficina, hijos IN OUT tAgentes );

     --Inserta un agente
     FUNCTION inserta_agente ( mi_agente agente )
     RETURN NUMBER;

     --Inserta una familia
     FUNCTION inserta_familia( mi_familia familia )
     RETURN NUMBER;

     --Inserta una oficina
     FUNCTION inserta_oficina ( mi_oficina oficina )
     RETURN NUMBER;

     --Borramos una oficina
     FUNCTION borra_oficina( id_oficina NUMBER )
     RETURN NUMBER;

     --Borramos una familia
     FUNCTION borra_familia( id_familia NUMBER )
     RETURN NUMBER;

     --Borramos un agente
     FUNCTION borra_agente( id_agente NUMBER )
     RETURN NUMBER;
END call_center;
/

CREATE OR REPLACE PACKAGE BODY call_center AS          --cuerpo
     --Implemento las funciones definidas en la especificación

     --Nos devuelve la oficina padre de un agente
     PROCEDURE oficina_padre( mi_agente agente, padre OUT oficina ) IS
          mi_familia familia;
     BEGIN
          IF (mi_agente.oficina IS NOT NULL) THEN
               SELECT * INTO padre FROM oficinas
               WHERE identificador = mi_agente.oficina;
          ELSE
               SELECT * INTO mi_familia FROM familias
               WHERE identificador = mi_agente.familia;
               oficina_padre( mi_familia, padre );
          END IF;
     EXCEPTION
          WHEN OTHERS THEN
               padre := NULL;
     END oficina_padre;

     --Nos devuelve la oficina padre de una familia
     PROCEDURE oficina_padre( mi_familia familia, padre OUT oficina ) IS
          madre familia;
     BEGIN
          IF (mi_familia.oficina IS NOT NULL) THEN
               SELECT * INTO padre FROM oficinas
               WHERE identificador = mi_familia.oficina;
          ELSE
               SELECT * INTO madre FROM familias
               WHERE identificador = mi_familia.familia;
               oficina_padre( madre, padre );
          END IF;
     EXCEPTION
          WHEN OTHERS THEN
               padre := NULL;
     END oficina_padre;

     --Nos da los hijos de una familia
     PROCEDURE dame_hijos( mi_familia familia, hijos IN OUT tAgentes ) IS
          CURSOR cHijos IS SELECT * FROM agentes
          WHERE familia = mi_familia.identificador;
          CURSOR cHijas IS SELECT * FROM familias
          WHERE familia = mi_familia.identificador;
          hijo agente;
          hija familia;
     BEGIN
          --inicializamos la tabla si no lo está
          if (hijos IS NULL) THEN
               hijos := tAgentes();
          END IF;
          --metemos en la tabla los hijos directos
          OPEN cHijos;
          LOOP
               FETCH cHijos INTO hijo;
               EXIT WHEN cHijos%NOTFOUND;
               hijos.EXTEND;
               hijos(hijos.LAST) := hijo;
          END LOOP;
          CLOSE cHijos;
          --hacemos lo mismo para las familias hijas
          OPEN cHijas;
          LOOP
               FETCH cHijas INTO hija;
               EXIT WHEN cHijas%NOTFOUND;
               dame_hijos( hija, hijos );
          END LOOP;
          CLOSE cHijas;
     EXCEPTION
          WHEN OTHERS THEN
               hijos := tAgentes();
     END dame_hijos;

     --Nos da los hijos de una oficina
     PROCEDURE dame_hijos( mi_oficina oficina, hijos IN OUT tAgentes ) IS
          CURSOR cHijos IS SELECT * FROM agentes
          WHERE oficina = mi_oficina.identificador;
          CURSOR cHijas IS SELECT * FROM familias
          WHERE oficina = mi_oficina.identificador;
          hijo agente;
          hija familia;
     BEGIN
          --inicializamos la tabla si no lo está
          if (hijos IS NULL) THEN
               hijos := tAgentes();
          END IF;
          --metemos en la tabla los hijos directos
          OPEN cHijos;
          LOOP
               FETCH cHijos INTO hijo;
               EXIT WHEN cHijos%NOTFOUND;
               hijos.EXTEND;
               hijos(hijos.LAST) := hijo;
          END LOOP;
          CLOSE cHijos;
          --hacemos lo mismo para las familias hijas
          OPEN cHijas;
          LOOP
               FETCH cHijas INTO hija;
               EXIT WHEN cHijas%NOTFOUND;
               dame_hijos( hija, hijos );
          END LOOP;
          CLOSE cHijas;
     EXCEPTION
          WHEN OTHERS THEN
               hijos := tAgentes();
     END dame_hijos;

     --Inserta un agente
     FUNCTION inserta_agente ( mi_agente agente )
     RETURN NUMBER IS
     BEGIN
          IF (mi_agente.familia IS NULL and mi_agente.oficina IS NULL) THEN
               RETURN operacion_no_permitida;
          END IF;
          IF (mi_agente.familia IS NOT NULL and mi_agente.oficina IS NOT NULL) THEN
               RETURN operacion_no_permitida;
          END IF;
          INSERT INTO agentes VALUES (mi_agente.identificador, mi_agente.nombre, mi_agente.usuario, mi_agente.clave, mi_agente.habilidad, mi_agente.categoria, mi_agente.familia, mi_agente.oficina );
          COMMIT;
          RETURN todo_bien;
     EXCEPTION
          WHEN referencia_no_encontrada THEN
               ROLLBACK;
               RETURN padre_inexistente;
          WHEN no_null THEN
               ROLLBACK;
               RETURN no_null_violado;
          WHEN DUP_VAL_ON_INDEX THEN
               ROLLBACK;
               RETURN elemento_existente;
          WHEN OTHERS THEN
               ROLLBACK;
               RETURN SQLCODE;
     END inserta_agente;

     --Inserta una familia
     FUNCTION inserta_familia( mi_familia familia )
     RETURN NUMBER IS
     BEGIN
          IF (mi_familia.familia IS NULL and mi_familia.oficina IS NULL) THEN
               RETURN operacion_no_permitida;
          END IF;
          IF (mi_familia.familia IS NOT NULL and mi_familia.oficina IS NOT NULL) THEN
               RETURN operacion_no_permitida;
          END IF;
          INSERT INTO familias VALUES ( mi_familia.identificador, mi_familia.nombre, mi_familia.familia, mi_familia.oficina );
          COMMIT;
          RETURN todo_bien;
     EXCEPTION
          WHEN referencia_no_encontrada THEN
               ROLLBACK;
               RETURN padre_inexistente;
          WHEN no_null THEN
               ROLLBACK;
               RETURN no_null_violado;
           WHEN DUP_VAL_ON_INDEX THEN
               ROLLBACK;
               RETURN elemento_existente;
          WHEN OTHERS THEN
               ROLLBACK;
               RETURN SQLCODE;
     END inserta_familia;

     --Inserta una oficina
     FUNCTION inserta_oficina ( mi_oficina oficina )
     RETURN NUMBER IS
     BEGIN
          INSERT INTO oficinas VALUES (mi_oficina.identificador, mi_oficina.nombre, mi_oficina.domicilio, mi_oficina.localidad, mi_oficina.codigo_postal );
          COMMIT;
          RETURN todo_bien;
     EXCEPTION
          WHEN no_null THEN
               ROLLBACK;
               RETURN no_null_violado;
          WHEN DUP_VAL_ON_INDEX THEN
               ROLLBACK;
               RETURN elemento_existente;
          WHEN OTHERS THEN
               ROLLBACK;
               RETURN SQLCODE;
     END inserta_oficina;

     --Borramos una oficina
     FUNCTION borra_oficina( id_oficina NUMBER )
     RETURN NUMBER IS
          num_ofi NUMBER;
     BEGIN
          SELECT COUNT(*) INTO num_ofi FROM oficinas
          WHERE identificador = id_oficina;
          IF (num_ofi = 0) THEN
               RETURN elemento_inexistente;
          END IF;
          DELETE oficinas WHERE identificador = id_oficina;
          COMMIT;
          RETURN todo_bien;
     EXCEPTION
          WHEN OTHERS THEN
               ROLLBACK;
               RETURN SQLCODE;
     END borra_oficina;

     --Borramos una familia
     FUNCTION borra_familia( id_familia NUMBER )
     RETURN NUMBER IS
          num_fam NUMBER;
     BEGIN
          SELECT COUNT(*) INTO num_fam FROM familias
          WHERE identificador = id_familia;
          IF (num_fam = 0) THEN
               RETURN elemento_inexistente;
          END IF;
          DELETE familias WHERE identificador = id_familia;
          COMMIT;
          RETURN todo_bien;
     EXCEPTION
          WHEN OTHERS THEN
               ROLLBACK;
               RETURN SQLCODE;
     END borra_familia;

     --Borramos un agente
     FUNCTION borra_agente( id_agente NUMBER )
     RETURN NUMBER IS
          num_ag NUMBER;
     BEGIN
          SELECT COUNT(*) INTO num_ag FROM agentes
          WHERE identificador = id_agente;
          IF (num_ag = 0) THEN
               RETURN elemento_inexistente;
          END IF;
          DELETE agentes WHERE identificador = id_agente;
          COMMIT;
          RETURN todo_bien;
     EXCEPTION
          WHEN OTHERS THEN
               ROLLBACK;
               RETURN SQLCODE;
     END borra_agente;
END call_center;
/
```

Anexo.- Licencias de recursos.

Ningún recurso de fuentes externas que requiera citar explícitamente sus datos de licencia ha sido usado en esta unidad, por lo que este anexo queda vacío. Todos los recursos utilizados, de fuentes internas, se acogen al Aviso Legal de la plataforma.